

与单片机邂逅

--单片机入门导引

通信科技创新实验室

目录

工程一：流水灯设计	1
导引：用灵感去创造多彩的世界	1
1.1 实现流水灯	2
1.2 LED 灯的控制	2
1.3 让蜂鸣器响起来	11
1.4 基础知识回顾：	11
1.5 扩展与反思	16
反思与改错	19
工程二：数码管显示	20
引例：数码管是一种半导体发光器件，其基本单元是发光二极管。	20
2.1 点亮数码管	20
2.2 控制数码管	23
2.3 余辉效应	29
2.4 基础知识回顾：	31
2.5 拓展与反思	33
反思与改错	37
工程三：按键和矩阵键盘	38
导引：用手指点亮精彩	38
3.1 按键原理图	38
3.2 抖动	39
3.3 独立按键实例程序	41
3.4 矩阵键盘	43
3.5 基础知识回顾	44
3.6 扩展与反思	44
反思与改错	51
工程四：中断系统及定时器/计数器	52
引例：单片机的“一心二用”	52
4.1 中断（服务图解及 8051 结构图）	52
4.2 定时器/计数器原理	54
4.3 中断	54

4.4 用中断来做一个时钟（进一步了解中断）	57
4.5 中断即时应用——>更快的掌握中断	59
4.6 感悟	62
4.7 错误是最好的老师	62
4.8 几种中断的模板	65
反思与改错	69
工程五：串口通信及串口中断	70
导引：芯与芯的桥梁——串口	70
5.1 串口	70
5.2 异步通信与同步通信	70
5.3 串行通信的传输方向	71
5.4 串行口缓冲寄存器 SBUF	72
5.5 串行控制寄存器 SCON	73
5.6 波特率	74
5.7 串口初始化	74
5.8 串口中断	74
5.9 让我们来写第一个完整的串口通信 C 程序	76
5.10 串口的拓展	79
5.10 心德	81
6.1 错误是最好的老师	81
反思与改错	86
附录	87
KEIL 的安装与调试	87
1. KEIL 软件的安装	87
2 KEIL 软件的调试	96

工程一：流水灯设计

导引：用灵感去创造多彩的世界

在我们现代的日常生活中，总能够看到各种各样的灯。有的简单实用，给我们生活带来诸多的方便之处；有的色彩斑斓，给我们呈现美丽的夜景。然而，这些都离不开单片机。



来自 14 栋 XX 宿舍的饮水机，可通过单片机控制显示加热或保温。



当年玩过的老虎机，现在你也可以控制它的灯了。



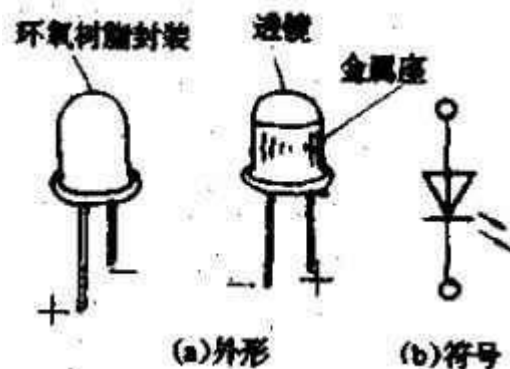
在 iPhone 之前可是“我”的时代（跑马灯可以通过单片机来实现）。



璀璨美丽的夜景，多彩多变令人沉醉其中的灯光都是通过单片机来控制。

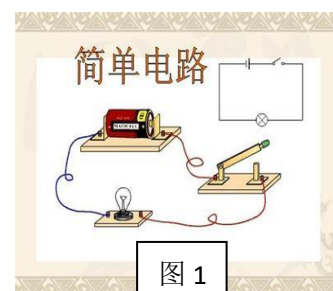
1.1 实现流水灯

1. 1. 1 发光二极管的构造与工作原理：



认识 led: Led (发光二极管) 灯是一种半导体发光器件, 十几毫安甚至几毫安的电流就能使 led 灯发出耀眼的光芒, 我们使用的单片机输出电流就只有几毫安, 于是当我们让单片机输出电流就能使 led 发光。

我们复习一下初中的物理, 点亮一颗小电灯, 于是我们需要一块电池, 几根导线, 一个开关, 然后我们把这些东西串联起来小灯泡肯定是不发光的 (图 1), 因为我们没有闭合开关, 先在我们看一下单片机的硬件连接图 (图 2)。图中可以看见开关被替换成了单片机, 在这里单片机被当成了开关, 想像一下单片机内部有几个开关, 而我们的目的就是写一个程序是开关合闭, 此外 led 灯是单向导电的要注意电流方向, 否则 led 灯就不会亮额, 初中高中知识我们就知道 led 说通俗点就是一个二极管, 反向通电的时候, 电阻相当于无穷大, 无法导通, (当然你可以加足够大的电压, 这样可以导通了, 但是你的 led 通过正向电流的时候就再也亮不了了, 发光二极管特性反向击穿后, 就损毁了, 所以大家以后做电路的时候一定要注意 led 正负级接反没有额。)



1.2 LED 灯的控制

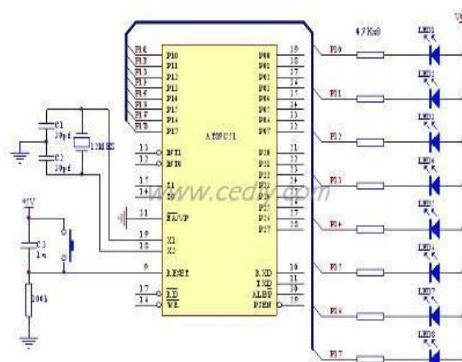
1.2.1 点亮一个 LED 灯

```

/*****
名称：点亮一个 LED
内容：第一次单片机之旅
*****/

```

```
#include <reg52.h> //8051 单片机特有的头文件
sbit LED = P0^0; // LED 为 P0 口 八个并联
void main() //主函数
{
    while(1)
    {
        LED = 0; //由单片机的原理 低电平工作 0 表示低， 1 表示高
    }
}
```



当然我们在编写程序的时候总会在链接编译的时候出现错误我们来看看这个程序出现错误的时候我们应该怎么办？

```
第一课.C(3): error C141: syntax error near 'void', expected ';'
第一课.c - 1 Error(s), 1 Warning(s).
```

如图 3 显示的是我的一个工程文件在编译的时候出现错误，这时候你就不可能将程序下载到 51 芯片里面，当然就不会让我们开发板上的 LED 亮起来。这到底是什么错呢？图 3 描述的是在你的 C 程序第三行有错，错误在 ‘void’ 附近，少了 ‘;’ 号，接下来我们调试程序（大家不要觉得调试程序很痛苦，当你调试多了，你就会有很多经验，有些错误一眼就知道在什么位置，有时候你写的程序，编译一下几十个错误，当然这不可能把你吓到，依次从上到下检查，一会儿就解决了）。

```
1 #include <reg52.h> //8051单片机特有的头文件
2 sbit LED = P0^0 // LED 为P0口 八个并联
3 void main() //主函数
```

很明显我们发现，箭头上面那句 `sbit LED = P0^0` 少了 ‘;’ 和错误类型一样当我们加上这个 ‘;’ 时候重新编译程序

```
#include <reg52.h> //8051单片机特有的头文件
sbit LED = P0^0; // LED 为P0口 八个并联
void main() //主函数
```

```
Build target 'Target 1'
linking...
Program Size: data=9.0 xdata=0 code=19
creating hex file from "第一课"...
"第一课" - 0 Error(s), 0 Warning(s).
```

现在程序已经编译成功，可以将 HEX 文件烧入芯片，就可以点亮 LED 了。

我认为这是第一次接触单片机，第一次写的程序，开启单片机之旅，就像学 C 语言一样 我们写的第一个程序，就是 “hello the word” .

思考：我们怎么写程序，让这个 LED 有频率的闪动？

1.2.2 让一个 LED 灯闪烁

经过上面的步骤，我们已经实行了流水灯一个部分了，虽然看起和流水灯差距还大，至少我们已经点亮了一个灯。

在这我们引入一个延时子函数

```
/******
****功能：延时
****入口参数：无
****出口参数：无
*****/
void Delaysms() //定义延时子函数
{
    uint i, j;

    for (i=1000; i>0; i--)
    {
        for (j=110; j>0; j--);
    }
}
```

有了这个延时函数仔细想想，如果将 1.1.1 的程序进行改动就行了

```

while(1)
{
    LED = 0;
    Delaysms(); //延时
    LED = 1;    //设置成高电平，LED 熄灭
    Delaysms();
}

```

组装完成的程序

```

01 #include <reg52.h> //8051单片机特有的头文件
02 #define uint unsigned int
03 sbit LED = P0^0;    // LED 为P0口 八个并联
04 void main()         //主函数
05 {
06     while(1)
07     {
08         LED = 0;
09         Delaysms(); //延时
10         LED = 1;    //设置成高电平，LED熄灭
11         Delaysms();
12     }
13 }
14 void Delaysms()      //定义延时子函数
15 {
16     uint i, j;
17
18     for (i=1000; i>0; i--)
19     {
20         for (j=110; j>0; j--);
21     }
22 }

```

如果是这样 我可以提前剧透给大家，编译链接的时候肯定会出现错误，接下来我们进行编译链接。

```

compiling 第一课.c...
第一课.C(1): warning C500: LICENSE ERROR (R208: RENEW LICENSE ID CODE (LIC))
第一课.C(9): warning C206: 'Delaysms': missing function-prototype
第一课.C(15): error C231: 'Delaysms': redefinition
第一课.C(22): error C231: 'Delaysms': redefinition
第一课.c - 2 Error(s), 2 Warning(s).

```

首先一个警告出现在第 9 行。（第一行的警告可以无视，那是我本身编译器的问题），第 9 行的意思说 识别不了 ‘Delaysms’ 这个类型到底是什么，其实我们也知道编译器就是个软件，只能识别二进制 0 和 1，我们只有通过这个语言所带有特定的规则才能让我们的程序跑起来，（我们的程序也是被编译器自动转换，这些都不是我们去了解的），接着在 15、22 行依次出现了这个问题，意思是读不懂 ‘Delaysms’，其实这问题和 C 语言调用函数一样。我们知道 C 语言是从上依次向下执行，从 main 函数去调用子函数，但是这个子函数怎么写才能避免上面的情况呢？我们知道子函数如果写在 main 函数的后面就要前面申明子函数名，如下图

/******

名称：一个闪烁 LED

内容：让一颗 LED 闪烁而不是一直亮或者一直熄灭

*****/

```
01 #include <reg52.h> //8051单片机特有的头文件
02 #define uint unsigned int
03 sbit LED = P0^0; // LED 为P0口 八个并联
04 void Delayms(); //子函数申明
05 void main() //主函数
06 {
07     while(1)
08     {
09         LED = 0;
10         Delayms(); //延时
11         LED = 1; //设置成高电平，LED熄灭
12         Delayms();
13     }
14 }
15 void Delayms() //定义延时子函数
16 {
17     uint i, j;
18
19     for (i=1000; i>0; i--)
20     {
21         for (j=110; j>0; j--);
22     }
23 }
```

接下来编译的时候就会通过，当然也可以这么写如下图

```
01 #include <reg52.h> //8051单片机特有的头文件
02 #define uint unsigned int
03 sbit LED = P0^0; // LED 为P0口 八个并联
04 void Delayms() //定义延时子函数
05 {
06     uint i, j;
07
08     for (i=1000; i>0; i--)
09     {
10         for (j=110; j>0; j--);
11     }
12 }
13 void main() //主函数
14 {
15     while(1)
16     {
17         LED = 0;
18         Delayms(); //延时
19         LED = 1; //设置成高电平，LED熄灭
20         Delayms();
21     }
22 }
```

其实这两种写法目的都是一样的让编译器知道‘Delayms’是什么东西，上图就是

main 函数在最后利用 C 语言特性程序是从上到下执行，所以不会存在错误。

这里通过调用函数，进行延时，这样我们就完成了目标让一个 LED 达到闪烁的效果

1.2.3 点亮八位 LED 灯

我们目标是一起点亮八个 LED

我们知道 P0 具有八个口 P0^0 到 P0^7，可能认真学的同学下列想法

```
sbit LED1 = P0^0; //定位 LED1 指向 P0^0 口
sbit LED2 = P0^1; //定位 LED2 指向 P0^1 口
sbit LED3 = P0^2; //定位 LED3 指向 P0^2 口
sbit LED4 = P0^3; //定位 LED4 指向 P0^3 口
```

。。。。。。如果说，这么做是对是错，我们可以明确的说，这是对的，非常对。

但是，大家想想这样是不是很麻烦呢？

于是我们在这引入进制运算

先列举一个程序

```
/******
```

名称：点亮一排 LED

内容：由宏定义端口利用十六进制一次性点亮一排 LED

```
*****/
```

```
#include <reg52.h> //8051 单片机特有的头文件
```

```
#define LED P0 // 自定义 P0 口为 LED
```

```
void main() //主函数
```

```
{
```

```
    while(1)
```

```
    {
```

```
        LED = 0x00//低电平工作 0 表示低，1 表示高
```

```
    }
```

```
}
```



这样写的效果就是 8 个灯全亮，为什么呢？

0x00 为前缀 0X 为 16 进制的标志，0x00 表示就是 0000 0000 刚刚八个位置从右向左对应 P0^0 到 P0^7。

在举个例子，如果讲 LED = 0x00 换成 LED = 0x11，表示的是 0001 0001 就是从右 向左第一个灯和第五个灯不亮，其余灯都亮。

思考：如果 LED = 0xff， 灯的亮灭情况呢？

1.2.4 让一位 LED 灯移位并实现流水灯效果

- 1) 可用专门的移位函数 `a=_crol_(a, b);`来实现。(第 `a` 位向右移动 `b` 个单位)
- 2) 也可以用数组定义所有 P0 口的点位，通过循环访问数组来点亮和关闭 LED 灯。

程序：

- 3) 还可以用位运算符进行

第一种

```
/******  
名称：一位 LED 灯移位实现流水灯  
内容：利用_crol_() 移位函数显示流水灯  
*****/  
  
#include <reg52.h>           //包含 51 单片机库函数头文件  
#include <intrins.h>         //包含定时器头文件  
#define uint unsigned int    //重新定义数据类型  
#define uchar unsigned char  
  
uint aa;                     //定义变量 aa 用于表示 P0 口的整体高低电  
平状态  
void Delayms();  
void main()  
{  
    aa = 0xfe;                //定义 aa 值为（二进制）1111 1110  
    while(1)  
    {  
        P0 = aa;              //将 aa 值赋给 P0 口  
        Delayms();             //使其状态保持一段时间  
        aa = _crol_(aa, 1);    //调用移位函数，使 aa 的二进制值向左一次移动  
一位  
    }  
}  
  
void Delayms()                //定义延时子函数  
{  
    uint i, j;  
  
    for (i=1000; i>0; i--)  
    {  
        for (j=110; j>0; j--);  
    }  
}
```

第二种：

```
/******  
名称：流水灯
```

内容：利用数组先定义控制 LED 灯的各种状态

```

*****/
#include <reg52.h>
#define uint unsigned int
uint aa[] = {0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f}; //将灯的不同状态分别定义成 8 组六进制数并存入数组中
void Delayms(); //延时子函数申明
void main()
{
    uint i = 0;
    while(1)
    {
        P0 = aa[i]; //定义 P0 口为 aa 数组中第 i 个元素的状态
        Delayms(); //延时一段时间
        i++; //改变数组中元素下标，当再次调用时，LED 灯的状态改变
        if (i == 8) //当所有状态都运行一次后应该回到最初的状态，才能得以无限循环
        {
            i = 0;
        }
    }
}

void Delayms() //延时子函数定义
{
    uint i, j;

    for (i=1000; i>0; i--)
    {
        for (j=110; j>0; j--);
    }
}

```

有时候手误可能出现以下错误，如下图

```

void main()
{
    uint i = 0;
    while(1)
    {
        P0 = aa[i]; //定义 P0 口为 aa 数组中第 i 个元素的状态
        Delayms(); //延时一段时间
        i++; //改变数组中元素下标，当再次调用时，LED 灯的状态改变
        if (i = 8) //当所有状态都运行一次后应该回到最初的状态，才能得以无限循环
        {
            i = 0;
        }
    }
}

```

然后编译就会出现如下图的警告

```
Build Output
compiling 第一课.c...
第一课.C(1): warning C500: LICENSE ERROR (R208: RENEW LICENSE ID CODE (LIC))
第一课.C(13): warning C276: constant in condition expression
第一课.c - 0 Error(s), 2 Warning(s).
```

为什么只是警告而不是错误呢（第一个警告无视 我本身编译器的问题）因为 `if(i = 8)` 的写法并没有不符合程序写法规则，只是最后你把程序烧入芯片，会发觉效果并不是你所写程序的效果，所以有时候警告也要注意。当把第 13 行的改成 `if(i == 8)` 程序就会正常运行。

第三种：

```
/******
```

名称：流水灯

内容：利用位运算实现流水灯

```
*****/
```

```
#include <reg52.h>
```

```
#define uint unsigned int
```

```
#define LED P0
```

```
void Delaysms(); //延时子函数申明
```

```
void main()
```

```
{
```

```
    uint i;
```

```
    while(1)
```

```
    {
```

```
        LED=0x01;
```

```
        for(i=0;i<8;i++) //for 循环
```

```
        {
```

```
            LED=~(LED<<i); //先向左移动 i 位 然后在进行取反
```

```
            Delaysms(); //延时 一定要延时才看得出来效果
```

```
        }
```

```
    }
```

```
}
```

```
void Delaysms() //延时子函数定义
```

```
{
```

```
    uint i, j;
```

```
    for (i=1000; i>0; i--)
```

```
    {
```

```
        for (j=110; j>0; j--);
```

```
    }
```

```
}
```

对 while（1）里面程序进行讲解，由于最先给 P0 口赋值是 0X01 也就是第一个灯不亮，其余全亮，但是这并不影响整个流水灯的运行，因为执行一条程序很快 肉眼根本就分辨不了，所以直接会看到第一个灯亮，其余全部熄灭，然后依次第二个灯亮，其余全灭。。。。

1.3 让蜂鸣器响起来

蜂鸣器的开启：

```
/******
```

名称：蜂鸣器

内容：让蜂鸣器发声

```
*****/
```

```
#include<reg52.h>
```

```
sbit Beep=P2^4;          //将 P2^4 口定义一个变量名字 Laba
```

```
void main(void)
```

```
{
```

```
    while(1)
```

```
    {
```

```
        Beep=0;          //将喇叭所在位置的三极管基极所连接的 P2^4 口电平拉低
```

```
    }
```

```
}
```

这个其实和点亮一个 LED 没什么区别。

思考：如何写程序让 LED 从右向左依次流动，LED 向左移动一次，蜂鸣器响一次？

提示：

```
    for(i=0;i<8;i++)
```

```
        //for 循环
```

```
    {
```

```
        LED=~(LED<<i);          //先向左移动 i 位 然后在进行取反
```

```
        Delayms();              //延时 一定要延时才看得出来效果
```

```
        Beep=0; //蜂鸣器响
```

```
        Delayms();
```

```
        Beep=1;
```

```
    }
```

利用程序从上到下依次执行 可以到达这个效果，不过还有没有其它更好的方法呢？

1.4 基础知识回顾：

1.4.1 延时函数：用延时函数来做延时，它不精确。

主要原理：机器在运行程序的时候，每执行一条语句，会用一定时间。因此我们可以利用机器循环执行命令来达到延时的效果

For 型：

```
/******  
****功能：延时 z 毫秒  
****入口参数： unsigned char z  
****出口参数： 无  
*****/  
void delay (unsigned char z)  
{  
    int i, j;  
  
    for (i=0; i<z; i++)  
    {  
        for (j=0; j<110; j++)//根据经验得出，没每执行 110 条语句，延时 1 毫秒。  
        ;                      //这个程序总共执行 z*110 次语句，延时 z 毫秒。  
    }  
}
```

while 型：

```
void delay (unsigned char z)  
{  
    while(z)  
    {  
        z--;  
    }  
}
```

1.4.2-crol-函数：crol_带循环左移

参数的设置：_crol_(unsigned char, unsigned char);

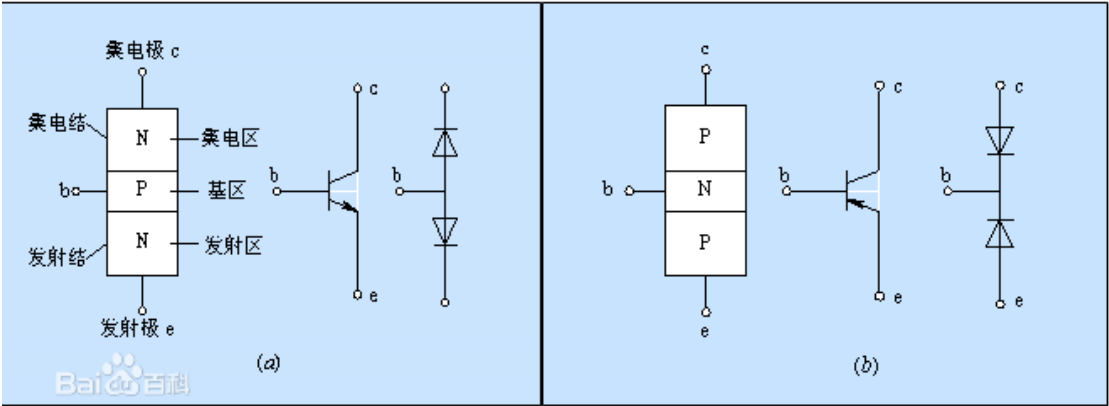
例如：_crol_(0x01,1);是 0000 0001 向左移动一位，则变为 0000 0010。对-crol-函数加上循环效果，就可以实现流水灯。

1.4.3 高电平：

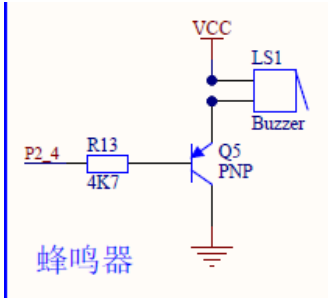
就是与低电平相对的高电压，是电工程上的一种说法。(有个系统了解就行，这些都会在以后的课程中学到，有兴趣的同学可以提前自学数电模电)

数字逻辑电路中的说法，在逻辑电路中，低电平表示 0，高电平表示 1。一般规定低电平为 0~0.25V，高电平为 3.5~5V。

1.4.4 三极管的简单了解:



三极管分 PNP 型和 NPN 型。
对于 PNP 型的三极管，当发射极与基极电势差大约大于 0.7V。三极管作用为放大。(由于制造工艺不同和使用材料不同，以后大家还会见着许多不同的三极管包括它的导通电压也会变化)
例如：当 P2.4 脚供低电平，三极管导通，蜂鸣器响。



1.4.5 P0 口端的整体定义与应用:

P0 口就是 P0.0 到 P0.7 的总和。
对 P0 口的赋值：P0=0xfe，0xfe 是一个十六进制的数，转化为二进制就是 1111 1110，刚好八位数，对应 P0 口的八个位置。这样就可以用一条语句对整个 P0 口操作了。

P0.0	P0.1	P0.2	P0.3	P0.4	P0.5	P0.6	P0.7
1	1	1	1	1	1	1	0

1.4.6 二进制

和十六进制的互相转换：
二进制为“逢二进一，借一当二”，十六进制为“逢十六进一，借一当十六”。

十六进制	二进制	十进制
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

这个码表其实也不用记，当你用多了，你就很熟了。

同时，使用计算器也可以进行快速的转化。



规律：一般把四个二进制数放在一起转换成一个十六进制数，转换时先把二进制数转换成十进制数，再把十进制数转换成十六进制数。

1.4.7 位运算符

运算符	含义
&	按位与
	按位或
^	按位异或
~	取反
<<	左移
>>	右移

位运算符对字节或字内的二进制数位进行测试、抽取、设置或移位等操作。操作对象不能是 float, double, long double, 只能是 char , int, unsigned int 类型

例如:

1) 按位与 &:

0000 1111 & 0000 0001 0000 0001	可以知道都是 1 与出来才是 1, 其 余的都是 0.
---------------------------------------	-----------------------------------

2) 按位或 |:

0000 1111 0000 0001 0000 1111	可以看出只要有 1, 或出来的数都 是 1。
---------------------------------------	------------------------------

3) 按位异或 ^:

0000 1111 ^ 0000 0001 0000 1110	相同的异或出来 都是 0, 不同的是 1
---------------------------------------	----------------------------

4) 按位取反 ~:

~ 0000 1111 1111 0000	顾名思义, 取反 就是取反
--------------------------	------------------

5) 左移 <<:

0000 1111 << 1 0001 1110	列子表示的是向 左移一位, 移动 后在右边自动补 0
-----------------------------	-------------------------------------

6) 右移 >>:

0000 1111 >> 1 0000 0111	列子表示的是向 右移一位, 移动 后在左边边自动 补 0
-----------------------------	---------------------------------------

1.5 扩展与反思

1.5.1 扩展:

注意：该部分较难，多多参考知识表来理解程序的每一句。
丰富多彩的流水灯

实验一：流水灯——碰撞

实验描述：将 led 灯分为两边，两边运动由中间对称，产生碰撞的效果。

实验分解：1) 将 led 灯分为两边：

左： ledl

右： ledr

2) 将左右流水灯初始化

ledl = led | 0x0f (|为 bit 位为 1 时有效，即把右 4 位全置为 1)

ledr = led | 0xf0

3) 左灯 (ledl) 向右，右灯向左

ledl = (ledl >> 1) | 0x80; (ledl >> 1 为把左半边亮着的灯向右移动一位，|为 1 有效，即补位位置为 1)

ledr = (ledr << 1) | 0x01;

4) 因为是对移，则碰撞后 ledl, ledr 运动交换即可。

temp = ledl;

ledl = ledr;

ledr = temp;

思考方法总结：对称运动——>将其分为两部分来做。

完整程序展示

实验一：流水灯——碰撞

/*

名称：流水灯——碰撞
内容：利用位运算控制流水灯进行碰撞

*/

名称：流水灯——碰撞

内容：利用位运算控制流水灯进行碰撞

*/

#include <reg52.h>

void Delay_ms(int n);

void main(void)

{

unsigned char led, ledl, ledr, temp;

led = 0x7e;

ledl = led | 0x0f;

ledr = led | 0xf0;

while (1)

{

```

        P0 = led;
        Delay_ms(50);
        if (led == 0xe7)
        {
            temp = ledr;
            ledr = ledl;
            ledl = temp;
        }
        ledl = (ledl >> 1) | 0x80;
        ledr = (ledr << 1) | 0x01;
        led = (ledl & ledr);
        if (led == 0x7e)
        {
            temp = ledr;
            ledr = ledl;
            ledl = temp;
        }
    }
}

void Delay_ms(int n)
{
    int i;
    while(n--)
    {
        for (i = 0; i < 1000; i++);
    }
}

```

实验二：流水灯——叠加

方法：又多次不同的初态，初态与前次的结束状态相同。

1) 保留前次的结束状态

```

        if (((led >> 1) | 0x80) == temp)
        {
            temp = led;
        }

```

2) 平移时，右边边界会移除，则采用初态合并法

```

        Led = ((led >> 1) | 0x80) | & temp;

```

思考方法总结：

每次变化都有不变的东西则<1>用一个变量储存不变的。<2>采用 & 或 | 获取不变的。

/******

名称：流水灯——叠加

内容：利用位运算对流水灯叠加效果

*****/

#include <reg52.h>

```

void Delay_ms(int n);
void main(void)
{
    unsigned char led, temp;
    led = 0x7f;
    temp = 0xff;
    while (1)
    {
        P0 = led;
        if (((led >> 1) | 0x80) == temp )
        {
            temp = led;
            led = (led & 0x7f);
            P0 = led;
            //Delay_ms(50);
        }
        else
        {
            led = ((led >> 1) | 0x80) & temp;
            Delay_ms(50);
        }
        if (led == 0x00)
        {
            P0 = led;
            Delay_ms(50);
            led = 0xff;
            temp = 0xff;
            P0 = led;
            Delay_ms(50);
            led = 0x7f;
        }
    }
}

void Delay_ms(int n)
{
    int i;
    while(n--)
    {
        for (i = 0; i < 1000; i++);
    }
}

```

1.5.2 反思

点亮 LED 和蜂鸣器为学习单片机的最基础的操作，其工作方式非常多，这是一个提高学习单片机兴趣的好开始。

反思与改错

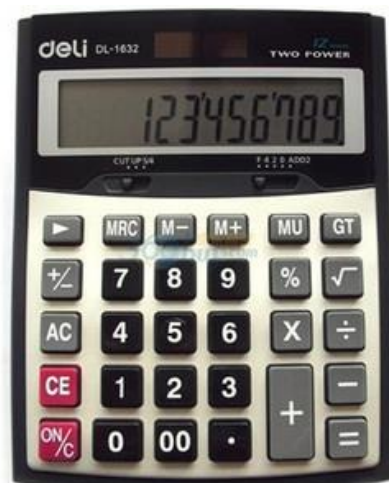
工程二：数码管显示

引例：数码管是一种半导体发光器件，其基本单元是发光二极管。

数码管也称 LED 数码管，颜色有红，绿，蓝，黄等几种。led 数码管广泛用于仪表，时钟，车站，家电等场合。



图一：家里面太阳能的显示屏



图二：计算器显示的数字由数码管来显示

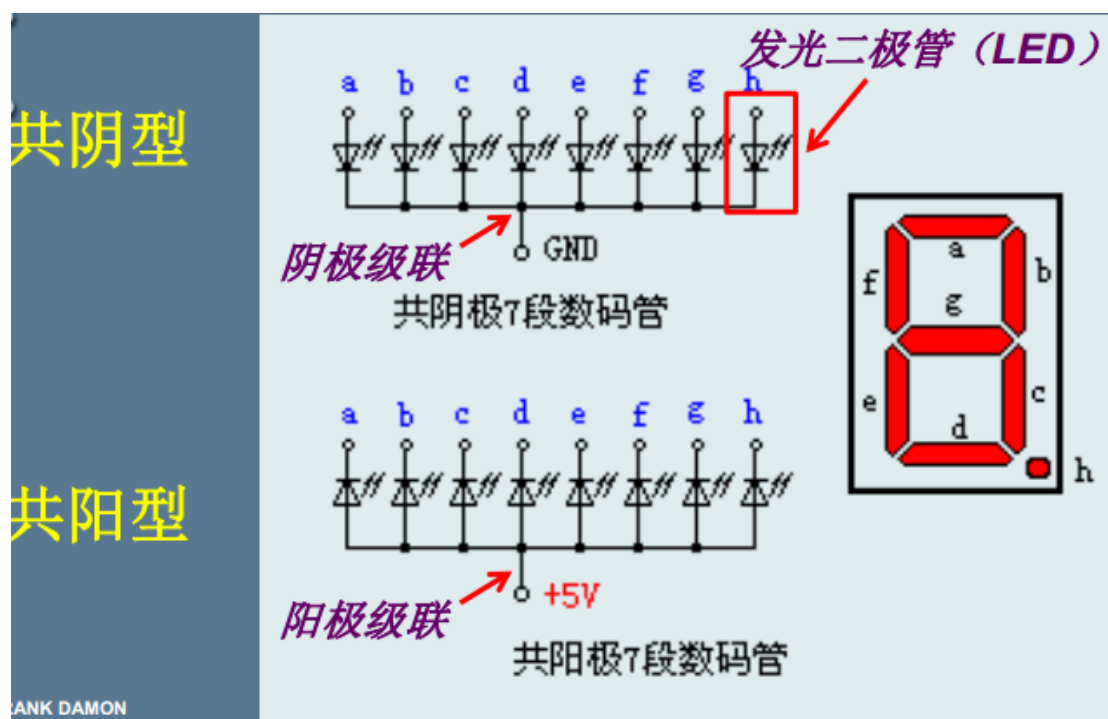


图三：单片机来自己做一个秒表显示

2.1 点亮数码管

2.1.1 发光二极管的构造与工作原理

数码管分共阴型和共阳型。共阴型：阴极连在一起接地；共阳型：阳极连在一起接电源正极。（图四）

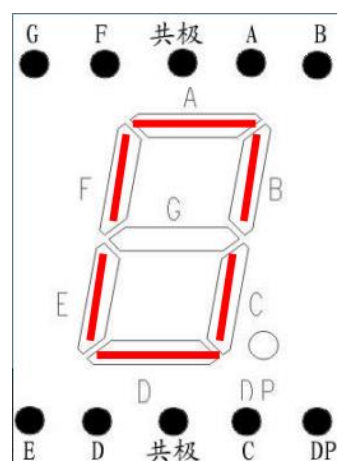


2.1.2 共阴型显示原理

阴极为数码管的公共端，按照发光二极管的原理，当阳极接电源正极，阴极接电源负极，发光二极管点亮。换句话说，共阴极的数码管，当段选引脚 a,b,c,d,e, f,g 脚分别接到电源的正极，而 GND 脚接的电源负极，7 个发光二极管同时点亮，并且显示“8”还有注意的是右下角“DP”或“H”是小数点。共阳二极管取反就是了。

例如：假设右边（图五）是一个共阴极的数码管，公共端已经接到低电平。段选管脚:DP,G,F,E,D,C,B,A 连接到单片机 P1 端口 7,6,5,4,3,2,1,0 管脚。

DP	G	F	E	D	C	B	A
7	6	5	4	3	2	1	0
0	0	1	1	1	1	1	1



如果要显示数字“0”需要点亮 A,B,C,D,E,F 六个 LED，即连接这六个 LED 的引脚输出高电平，其余为低电平。（共阳管取反即可）

2.1.3 数码管码表的使用方法

将码表定义在一个数组之中，型如：

```
code unsigned char LedHexCode[]={0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e}; //共阳数码管码表  
调用方式,例如: P1=LedHexCode[0]显示数字 0;
```

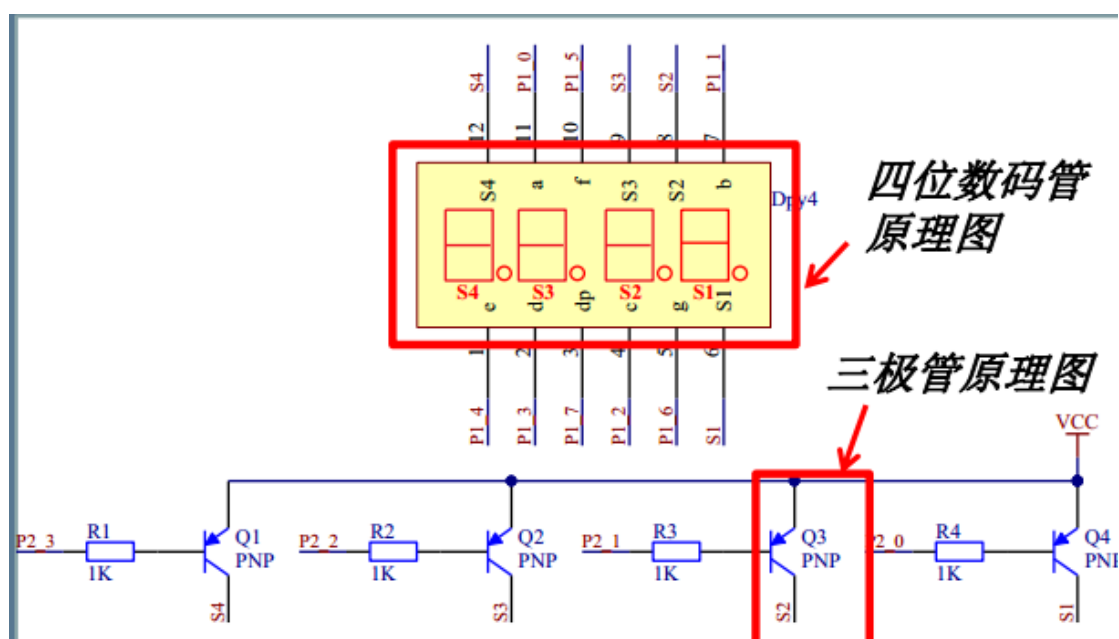
2.1.4 共阳管与共阴管码表参考:

字符	0	1	2	3	4	5	6	7
共阴型	0X3F	0X06	0X5B	0X45	0X66	0X6D	0X7D	0X07
共阳型	0XC0	0XF9	0XA4	0XB0	0X99	0X92	0X82	0XF8
字符	8	9	A	B	C	D	E	F
共阴型	0X7F	0X6F	0X77	0X7C	0X39	0X5E	0X79	0X71
共阳型	0X80	0X90	0X88	0X83	0XC6	0XA1	0X86	0X8E

那怎么使用码表呢，我们仔细想想前面写流水灯的程序的时候，其实就运用的就是码表，同时给八个口赋值，就是所谓的一个位对应一个点。

举个例子:我们给 P1 口赋值就是 $P1 = 0xc0$ ，(由于学院发的单片机开发板的数码管是共阳的，所以使用共阳数码管码表)，简单来说，就是 P1 有八个口同时赋值必须使用十六进制进行赋值。

2.1.5 总原理图（图六）:



四位数码管公用八个段值引脚。每一位的公共端单独引出，四个公共端构成位选

引脚，共 12 个引脚。

2.2 控制数码管

2.2.1 简单的数码管显示程序

2.2.2 显示“1”

/******

名称：数码管简单显示

内容：使数码管显示“1”

时间：2014 年 7 月 1 日

*****/

```
#include<reg52.h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
#define duan P1
```

```
sbit weil=P2^0;
```

```
unsigned char
```

code

```
table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};
```

```
void delay(uint z);
```

/******

名称：主函数

内容：大循环，使显示一直进行

入口参数：无

出口参数：无

*****/

```
void main(void)
```

```
{
```

```
    while(1)
```

```
    {
```

```
        duan=table[1];
```

```
        weil=0;
```

```
        delay(1);
```

```
        weil=1;
```

```
    }
```

```
}
```

/******

名称：延时函数

内容：延时 Z 毫秒

出口参数：无

*****/

```
void delay(uint z)
{
    int x, y;
    for (x=100; x>0; x--)
        for (y=z; y>0; y--);
}
```

效果显示:



虽然这个程序产生的效果很简单，但是在编写程序的时候，需要很多的前置的工作，比如配置码表，定义数据的类型等，这些现在看似繁琐，但是在以后编写大程序的时候，这些繁琐过程体现的作用可以帮你节省很多时间的。

2.2.3 数码管 0~9 递增

从 1 稳定的跳到 9。

/*****

名称：数码管简单显示

内容：使数码管显示“0~9”

时间：2014 年 7 月 1 日

*****/

```
#include<reg52. h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
#define duan P1
```

```
sbit weil=P2^0;
```

unsigned	char	code
----------	------	------

table[] = {0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90};

```

unsigned int i;

void delay(uint z);
void display(uint i);
/*****
名称：主函数
内容：大循环，使显示一直进行
入口参数：无
出口参数：无
*****/
void main(void)
{
    while(1)
    {
        for(i=0;i<9;i++)
        {
            display(i);
            delay(100);//使显示的数字显示 100MS 再做下一个显示
        }
    }
}
/*****
名称：显示函数
内容：最基本的显示程序段
入口参数：i
出口参数：无
*****/
void display(uint i)
{
    duan=table[i];
    weil=0;
    delay(1);
    weil=1;
}
/*****
名称：延时函数
内容：延时 Z 毫秒
入口参数：z
出口参数：无
*****/
void delay(uint z)
{
    int x, y;
    for (x=100; x>0; x--)

```

```

        for (y=z; y>0; y--);
    }

```

在把这个程序烧入单片机后，我们可以看到的效果是每一个数字只是稍微闪一下就黑了，然后在一段时间后又跳成下一个数字。这个我们想象中的效果不一样！我们可以看到这段程序：

```

    for(i=0;i<9;i++)
    {
        display(i);
        delay(100); //使显示的数字显示 100MS 再做下一个显示
    }

```

是每一个数字显示后再做的延时，因此效果不一样。要达到我们想要的效果，可以思考在哪个地方把延时做长一点，就可以有想要的效果。

把大循环改成如下的：

```

    for(i=0;i<9;i++)
    {
        display(i);
    }

```

大循环只做循环显示，把每个显示分给每个 display 函数解决。

```

/*****
名称：显示函数
内容：最基本的显示程序段
入口参数：i
出口参数：无
*****/
void display(uint i)
{
    duan=table[i];
    weil=0;
    delay(100);
    weil=1;
}

```

将位选的开关简短时间加长就能达到效果！

2.2.4 动态显示 0~9

这个程序的效果和上一个例子的效果一样，但是在这里没有用到数码的动态显示！大家可以先思考一下，如果不用动态显示，怎么能够达到效果呢？

```

/*****
名称：数码管简单显示
内容：使数码管显示“0~9”
时间：2014 年 7 月 1 日
*****/

```

```

#include<reg52.h>

```

```

#define uint unsigned int
#define uchar unsigned char
#define number P1

sbit Dpy0 = P2^0;
sbit Dpy1 = P2^1;
sbit Dpy2 = P2^2;
sbit Dpy3 = P2^3;

void Delay_Ms(uint tc); //函数延时 tc 毫秒

code uchar table[]={
0xc0 , 0xf9 , 0xa4 , 0xb0 , 0x99 , 0x92 ,
0x82 , 0xf8 , 0x80 , 0x90 , 0x88 , 0x83 ,
0xc6 , 0xa1 , 0x86 , 0x8e}; //共阳数码管码表

/*****
名称：主函数
内容：大循环，使显示一直进行
入口参数：无
出口参数：无
*****/
void main(void)
{
    uchar i;

    Dpy0 = 0;

    while(1)
    {
        for (i = 0; i <= 8; i++)
        {
            number = table[i]; //把定义的数组里第 i 号位置的值赋给 P1 口
            Delay_Ms(400);
        }
    }
}

/*****
名称：延时函数
内容：延时 Z 毫秒
入口参数：tc
出口参数：无
*****/

```

```

*****/
void Delay_Ms(uint tc) //函数延时 tc 毫秒
{
    uint i;

    while(tc != 0)
    {
        for(i = 0; i<122; i++)//执行 122 次（经验值）空循环将耗时 1 毫
        {

        }

        tc--;
    }
}

```

数码管的动态显示的要点就在于位选的开关和延时的控制！而在这个程序里面，我们没有用到位选的开关，单纯的给数码管的段选赋值就 OK 了。同时在这个程序里面我们可以看到不一样的延时函数：

/*****

名称：延时函数

内容：延时 Z 毫秒

入口参数：tc

出口参数：无

*****/

```

void Delay_Ms(uint tc) //函数延时 tc 毫秒
{
    uint i;

    while(tc != 0)
    {
        for(i = 0; i<122; i++)//执行 122 次（经验值）空循环将耗时 1 毫
        {

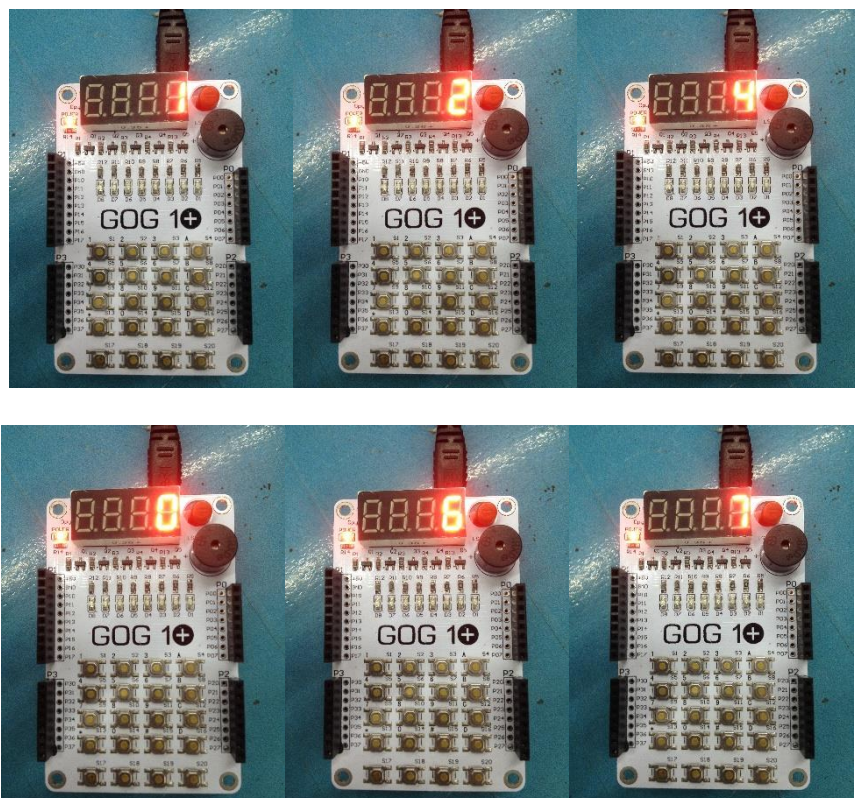
        }

        tc--;
    }
}

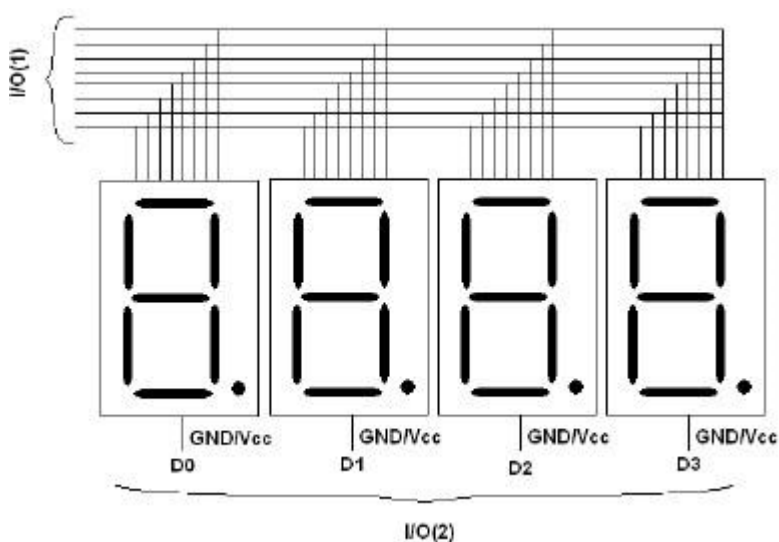
```

延时函数要达到延时的效果就行了，让单片机一直做事就可以延时！所以延时也不是只有一种方法。大家还可以发散一下，看看能不能写出其他写法的延时函数！

下面是这个两个显示程序的一些效果图：



2.3 余辉效应



产生原因：由于此 51 单片机数码管部分的接口电路是把所有数码管的 8 个段选 a-h 同名端连在一起，而每一个显示器的公共极 COM 是各自独立地受 I/O 线控制。CPU 向段位送出字形码时，所有数码管接收到相同的字形码，但究竟是哪个数码管亮呢，则取决于 COM 端，而这一端是由 I/O 控制的，所以我们就能够自行决定何时显示哪一位了。但是，如果我们先决定了位选再决定了段选，则在数码管的动态扫描中，若第一位显示数字 1，关闭第一位后，我们首先打开了第二位的位选，此时 CPU 仍然保持了之前数字 1 的段选，则此时第二位的

位选就会显示数字 1，而程序紧接着才会改变段选，所以我们总是能看到后一位选会有前一位选数字的残影，此现象简称余辉效应。（如图七为此 51 单片机的数码管电路图）解决方案：最简单的解决方案就是我们先选定并发送段选信号，然后再决定并发送位选信号。

2.3.1 有余辉效应:

/******

名称：数码管余晖效应

内容：使数码管产生余晖

时间：2014 年 7 月 1 日

*****/

#include<reg52.h>

#define uchar unsigned char

#define uint unsigned int

#define duan P1

sbit weil=P2^0;

unsigned char code

table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};

unsigned int i;

void delay(uint z);

void display(uint i);

/******

名称：主函数

内容：大循环，使显示一直进行

入口参数：无

出口参数：无

*****/

void main(void)

{

while(1)

{

display(i);

}

}

/******

名称：显示函数

内容：用数码管显示

入口参数：i

出口参数：无

*****/

void display(uint i)

{

```

        weil=0;
        duan=table[i];
        delay(1);
        weil=1;
    }
/*****
名称：延时函数
内容：延时 Z 毫秒
入口参数：z
出口参数：无
*****/
void delay(uint z)
{
    int x, y;
    for (x=100; x>0; x--)
        for (y=z; y>0; y--);
}

```

2.3.2 无余辉效应：

```

#include<reg52.h>
#define duan P1
uchar code table[] = {0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82,
0xf8, 0x80, 0x90};
sbit weil=P2^0;
void display(unsigned char num)
{
    duan=table[0]; //在显示之前就给数码管的段赋值
    weil=0;
    delayms();
    weil=1;
}

```

2.4 基础知识回顾：

2.4.1 C-51 的运算符

C-51 的运算符与 C 语言基本相同：

+ - * /	(加 减 乘 除)
> >= < <=	(大于 大于等于 小于 小于等于)
== !=	(测试等于 测试不等于)
&& !	(逻辑与 逻辑或 逻辑非)
>> <<	(位右移 位左移)
&	(按位与 按位或)
^ ~	(按位异或 按位取反)

位运算规则—与 &
与：参与运算的两个运算量，逐位“与”

如：3&5 3=00000011
 (8) 5=00000101
 00000001

如：八进制数60与17或运算

60=00110000
(1) 17=00001111
 00111111

0&0=0
0&1=0
1&0=0
1&1=1

0 0=0
0 1=1
1 0=1
1 1=1

位运算规则—或 |
或：参与运算的两个运算量逐为“或”

位运算规则—异或 ^
异或：0^0=0, 1^1=0, 0^1=1, 1^0=1

●异或：0^0=0, 1^1=0, 0^1=1, 1^0=1
如：

00111001
(^) 00101010
 00010011

位运算规则—取反 ~
取反：按位将 0 变 1, 1 变 0

如：将00000000000010101取反

↓

11111111111101010

位运算规则—移位 <<
左移(<<)：高位溢出，低位补 0

每移动1位，相当于乘以2

01111111 11111110
(127) <<1 (254)

右移(>>)：低位舍弃，高位补 0 或 1
每移动 1 位，相当于除以 2
无符号数高位补 0（逻辑右移）
带符号数高位补 1（逻辑右移）

00001111 00000011
(15) >>2 (3)

2.4.2 C—51 的基本数据类型

类型	符号	关键字	所占位数	数的表示范围
整型	有	(signed) int	16	-32768~32767
		(signed) short	16	-32768~32767
		(signed) long	32	-2147483648~2147483647
	无	unsigned int	16	0~65535
		unsigned short int	16	0~65535
		unsigned long int	32	0~4294967295
实型	有	float	32	3.4e-38~3.4e38
	有	double	64	1.7e-308~1.7e308
字符型	有	char	8	-128~127
	无	unsigned char	8	0~255

2.5 拓展与反思

2.5.1 扩展实验： 流动的数码管

实验现象：led 沿着 4 位数码管的周围依次循环闪亮形成一个环形流动光圈。

实验思想：

1. 把数码管要显示的样状的段码找出来。
2. 编写基本的数码管显示函数，用定时器扫描法。
3. 对循环进行分析。

1)在两端数码管显示会显示 4 段，而中间会显示 1 段。这样我们通过用选择语句将显示分为了两个不分。

2) 数码管是循环显示的所以应该分为位选的加和减，这里使用 count 标记法进行了分段。

4.这样我们就完成了数码管的流水灯了。

/******

名称：数码管的拓展应用

内容：使用数码管的段来形成跑马灯效果

时间：2014 年 7 月 1 日

*****/

#include <reg52.h>

unsigned char Duanma[] = {0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xfe};

sbit Dpy0 = P2^0;

sbit Dpy1 = P2^1;

sbit Dpy2 = P2^2;

sbit Dpy3 = P2^3;

/******

名称：延时函数

内容：延时 Z 毫秒

入口参数：n

出口参数：无

*****/

void Delay_ms(unsigned char n)

```

{
    unsigned char i;
    while (n--)
    {
        for (i = 0; i < 122; i++);
    }
}
/*****
名称：显示函数
内容：最基本的显示程序段
入口参数：n i
出口参数：无
*****/
void Display(unsigned char n, unsigned
char i)
{

    P1 = 0xff;
    P2 = 0xff;

    switch (n)
    {
        case 0: Dpy0 = 0;
                break;
        case 1: Dpy1 = 0;
                break;
        case 2: Dpy2 = 0;
                break;
        case 3: Dpy3 = 0;
                break;
    }

    P1 = Duanma[i];
    Delay_ms(50);
    P2 = 0xff;
}
/*****
名称：中断的配置
内容：对要实用的中断进行配置，达
到设置的点便进入中断
入口参数：无
出口参数：无
*****/

```

```

void InitTimer0(void)
{
    TMOD |= 0x0f;
    TMOD &= 0xf2;

    TH0 = 0x06;
    TL0 = 0x06;

    EA = 1;
    ET0 = 1;
    TR0 = 1;
}
/*****
名称：主函数
内容：大循环，使显示一直进行
入口参数：无
出口参数：无
*****/
void main (void)
{
    InitTimer0();
    while (1)
    {

    }
}
/*****
名称：中断 1
内容：达到时间，进入中断
入口参数：无
出口参数：无
*****/
void Break_Timer0 (void) interrupt 1
{
    static unsigned char d = 3, i, count;

    if (d == 0 )
    {
        for (i = 0; i < 3; i++)
        {
            Display(d, i);
        }
    }
}

```

```

count = 1;
}

if (d == 3 )
{
    for (i = 4; i < 7 ; i++)
    {
        Display(d, i);
    }

    i = 0;

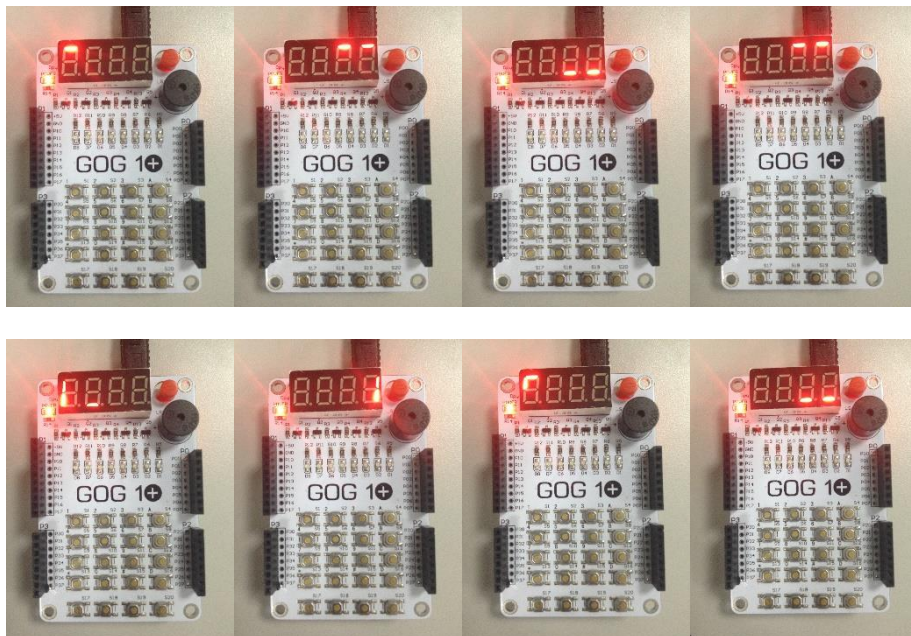
```

```

count  = 0;
}
Display(d, i);
if (count == 0)
d--;
if (count == 1)
d++;
}

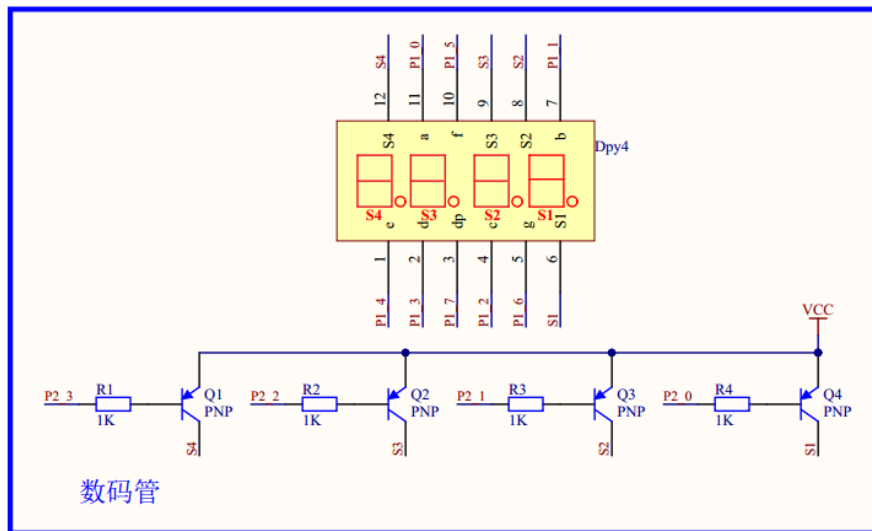
```

在这个程序里面涉及到了中断的应用，这是单片机的一个很重要很实用的一个部分，可以说是它的精髓所在。之所以在这里提出来，是想让大家有个提前学习和自学的意识。大学时间很紧的！下面是一些效果图：



2.5.2 反思

数码管的运行原理稍微复杂，理解起来可能会很费力。因此在学习这部分的时候要注意每一个细小的知识点，切忌直接看重点！大家也会感受到，在学习数码管的时候比学习流水灯上了一定的难度。大家可以看看开发板的原理图的 PDF，在里面会有详细的介绍板子的电路。



开发板是用三极管来作为开关的，还有一种操控数码管的方法就是用锁存器。用锁存器的时候就可以免去数码管麻烦的动态显示，它可以将要显示的段码锁住！大家可以先去自行了解了解~~

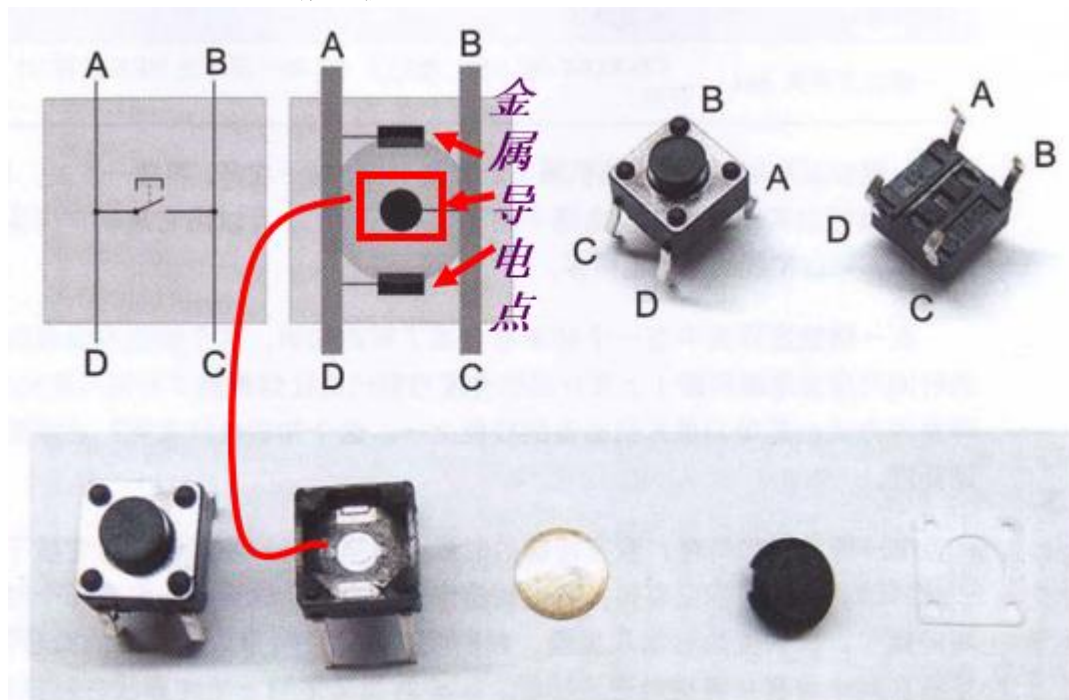
反思与改错

工程三：按键和矩阵键盘

导引：用手指点亮精彩

在我们这个大信息时代，生活中常常需要我们记住各种各样的密码以及使用各种各样的密码，甚至在使用简单的计算器等工具时，我们都需要进行输入操作，而搭载输入操作功能的基本器件就是我们的按键，在本章学习中我们主要学习由单片机控制的独立按键操作以及矩阵键盘的操作。如图所示就是我们常用的ATM机的输入键盘，这就是一个简单的4X4的矩阵键盘，那这些形形色色的按键具体是怎么读入数据信号来实现特定功能的呢？本章将告诉你矩阵键盘和独立按键的使用与应用。

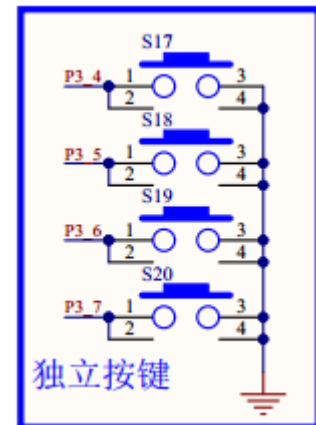
下面是实验室常用的微动按键实物图：



3.1 按键原理图

3.1.1 独立按键

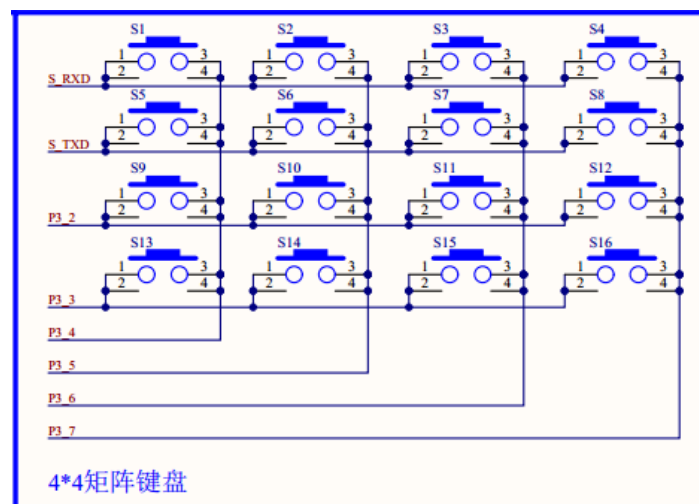
按键其实就是我们的开关原理其实很简单按下按键的过程就是把两个金属片接触在一起的过程。我们的开发板上有独立按键这就是独立按键的原理图，接法就是把开关的一端接到地线上，另外一端分别接在单片机的 $Px.x$ 口上，这个接法程序写起来相对简单。



3.1.2 矩阵按键

但是我想一下，89c52 这个单片机只有 32 个 i/o 口 最多只能接 32 个独立按键，如果我们想再多点接口怎么办呢，这就用到了一个非常巧妙的方法，那就是矩阵键盘，如图简述一下工作原理：

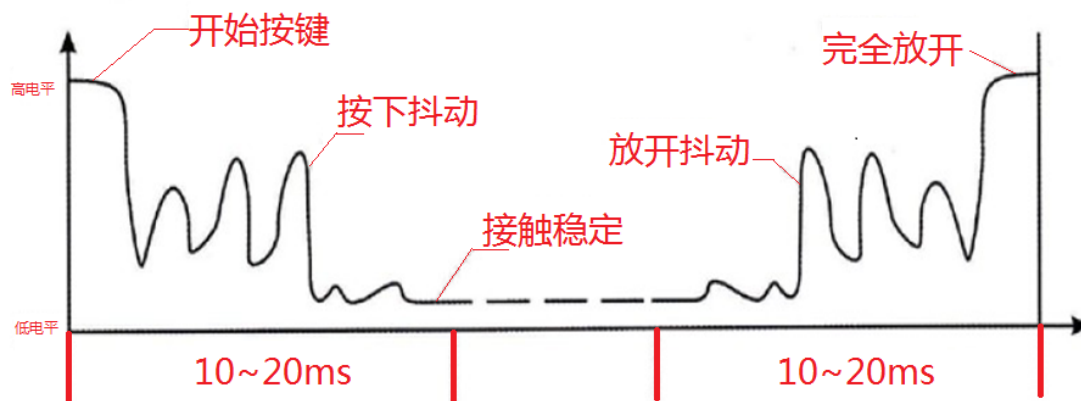
十六个按键就有十六个键值，用了 8 个 i/o 口，行选 4 个列选 4 个根据排列组合原理 $C_4^1 \times C_4^1 = 16$ 。按下去以后让单片机判断是哪两个 i/o 口被连接到了一起从而计算出键值



3.2 抖动

3.2.1 按键抖动原因

按键按下关键之一——抖动
用图形描述那个过程就是：



为什么会产生抖动呢？

由于单片机的工作频率比较高，在按键按下去的过程中和按键松开的过程中，都有可能产生连续的电平脉冲，从而使其 I/O 口发生连续的电平变化，为了防止程序误动作，就用软件来实现消抖动的动作。

3.2.2 如何消抖

由上图所示，当我们按下按键的时候 有一个抖动的过程，这个抖动怎么去解决呢？

小心翼翼的按下？这样的方法似乎不太好

接下来介绍一个简单的方法来消抖—>延时函数，按键抖动的时候，利用延时函数让单片机躲开这段时间。

延时函数的又来了

```
void Delay_Ms(unsigned int tc) //函数延时 tc 毫秒，子函数
{
    unsigned int i;
    while(tc != 0)
    {
        for(i = 0; i < 122; i++); //执行 122 次（经验值）空循环将耗
        时 1 毫秒
        tc--;
    }
}
```

接下来利用延时消抖动——核心程序

```
sbit KEY == P3^4;
sbit LED == P0^0;
if (KEY == 0) //按键按下
{
    Delay_Ms(100); //消抖
    if (KEY == 0) //消抖之后继续判断是否为低电平
    {
        LED = ~LED; //LED 灯改变状态
    }
}
```

```
}
```

以上就是一个简单的消抖程序。

3.3 独立按键实例程序

3.3.1 实例一：

独立按键控制 LED 灯的开关

```
/******  
/*使用独立按键点亮 LED 灯  
/*使用延时函数对按键进行消抖处理  
/******/  
#include<reg52.h>//头文件声明  
/*定义管脚*/  
sbit key=P3^4;//将 key 位定义为 p3.4  
/******  
/*利用空循环来实现延时  
/******/  
void delay(unsigned int a)//简单的延时  
{  
    for(;a>0;a--)  
    {  
  
    }  
}  
/******  
/*@program 主函数  
/*@bref  
/*消抖后判断按键的状态,  
/*如按键一次按下所有 LED 全亮, 再次按下就都不亮  
/******/  
void main(void)  
{  
    unsigned int count=0;  
    while(1)  
    {  
        if(key==0)  
        {  
            delay(15);//消抖  
            if(key==0)  
            {  
                P0=0x00;  
                count++;  
                if(count%2==0)  
                    P0=0xff;  
            }  
        }  
    }  
}
```

```

    }
}
}
}

```

3.3.2 实例二:

独立按键的伪中断功能(按键按下蜂鸣器响)

```

#include<stdio.h>
#define uchar unsigned char
sbit key = P3^4;
sbit beep = P2^4;
uchar code table[]={0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f, 0xff};

void main(void)
{
    uint i, a;
    while(1)
    {
        for (i=0; i<9; i++)
        {
            P0 = table[i];
            a = 10000;
            while(a--);
            if (key == 0)
            {
                beep = 0;
                a = 10000;
                while (a--);
                beep = 1;
            }
        }
    }
}

```

3.3.3 实例三:

独立按键控制蜂鸣器鸣笛的次数

```

/*****
/*使用延时来控制蜂鸣器的频率
/*用按键来选择不同的频率
*****/
/
#include<reg52.h> //头文件声明
/*****
*
/*管脚的定义
*****/

```

```

/
sbit sound=P2^4; //将 sound 位定为
P2.4
sbit key1=P3^4;
sbit key2=P3^5;
sbit key3=P3^6;
sbit key4=P3^7;
unsigned int i, j;
/*****
*

```

```

/*函数功能：延时形成 1600Hz 音频
*****
/
void delay1600(void)
{
    unsigned char n;
    for(n=0;n<100;n++);
}
/*****
*
函数功能：延时形成 800Hz 音频
*****
**/
void delay800(void)
{
    unsigned char n;
    for(n=0;n<200;n++);
}
void display(unsigned int a)
{
    for(j=0;j<a;j++)
    {
        for(i=0;i<830;i++)
        {
            sound=0; //P2.4 输出低电平
            delay1600();
            sound=1; //P2.4 输出高电平
            delay1600();
        }
    }
    for(i=0;i<200;i++)
    {
        sound=0; //P2.4 输出低电平
        delay800();
        sound=1; //P2.4 输出高电平
        delay800();
    }
}

```

3.4 矩阵键盘

3.4.1 矩阵按键的检测

- 1、将接矩阵键盘行的引脚设为高电平 0xf0
- 2、然后检测接矩阵键盘行中是否有低电平出现；若检测到某行为低电平后，确定是哪一行。

```

}
}
/*****
/*主函数，按键选择频率
*****
/
void main(void)
{
    while(1)
    {
        if(key1==0)
        {
            delay800(); //消抖
            if(key1==0)
            display(2);
        }
        if(key2==0)
        {
            delay800(); //消抖
            if(key2==0)
            display(4);
        }
        if(key3==0)
        {
            delay800(); //消抖
            if(key3==0)
            display(6);
        }
        if(key4==0)
        {
            delay800(); //消抖
            if(key4==0)
            display(8);
        }
    }
}

```

3、然后将列的引脚设为高电平 0xf0

4、检测列是否有高电平的列；若存在，则确定是哪一列。

5、这样就确定了一个按键的行和列坐标。

3.4.2 矩阵键盘检测核心代码：

```
#define KeyPad P3
```

```

.....
    KeyPad = 0x0f;    //设定初始电平
    注意 此处 0x0f
    if(KeyPad != 0x0f) //判断有无按键
    按下
    {
        delayms(20);//去个抖动
        if(KeyPad != 0x0f)//再次判
        断
        {
            a = KeyPad;// 临时的
            状态
        }
        KeyPad = 0xf0;//重新设定初
        始电平 注意此处 0xf0
        c = KeyPad;//临时的状态
        a |= c;//临时状态合并
        switch (a){//a 为前面计算好的按键
        状态值
            case 0xee:          b =
            1;                  break;
            case 0xed:          b =
            4;                  break;
            case 0xeb:          b =
            7;                  break;
            .....
            case 0x77:          b =
            13;                 break;
            default              :
            b = 0xff;           break;
        }//default 返回的 0xFF 代表
        错误值或无按键按下
    }

    /*******
    通过矩阵键盘来控制不同的 LED 的亮灭
    *****/

#include<reg52.h>
/*******
/*宏定义
/*管脚定义
*****/

#define uint unsigned int
#define uchar unsigned char
#define KEYS P3    //假设 P3 是矩阵键盘
#define LEDS P0

```

```

    }
    return (b);//返回真实键值
}

```

3.5 基础知识回顾

矩阵键盘扫描原理

方法一：

逐行扫描：我们可以通过高四位轮流输出低电平来对矩阵键盘进行逐行扫描，当低四位接收到的数据不全为 1 的时候，说明有按键按下，然后通过接收到的数据是哪一位为 0 来判断是哪一个按键被按下。

方法二：

行列扫描：我们可以通过高四位全部输出低电平，低四位输出高电平。当接收到的数据，低四位不全为高电平时，说明有按键按下，然后通过接收的数据值，判断是哪一行有按键按下，然后再反过来，高四位输出高电平，低四位输出低电平，然后根据接收到的高四位的值判断是那一行有按键按下，这样就能够确定是哪一个按键按下了。

3.6 扩展与反思

在学习了单片机的按键后，大家可以用做一些实际的事情。以下便是一个简单的例子，按下键后，通过对其定义的点亮 LED：

```

/*****
0   0   0   0   0 0 0 0 0x00
128 64  32  16  8 4 2 1
*****/

/*****
函数声明区
*****/

void Delay_Ms(uint tc); //函数延时 tc 毫秒
uchar KeyScan();

/*****
主函数
通过调用矩阵键盘子函数的返回值来点
亮不同的灯
*****/

int main()
{

```

```

uint a;
while(1)
{
    a = KeyScan();
    switch(a)
    {
        case 1:  LEDS = ~0x01;
                  break;
        case 2:  LEDS = ~0x02;
                  break;
        case 3:  LEDS = ~0x04;
                  break;
        case 4:  LEDS = ~0x08;
                  break;
        case 5:  LEDS = ~0x10;
                  break;
        case 6:  LEDS = ~0x20;
                  break;
        case 7:  LEDS = ~0x40;
                  break;
        case 8:  LEDS = ~0x80;
                  break;
        case 9:  LEDS = ~0x18;
                  break;
        case 10: LEDS = ~0x24;
                  break;
        case 11: LEDS = ~0x42;
                  break;
        case 12: LEDS = ~0x81;
                  break;
        case 13: LEDS = ~0x11;
                  break;
        case 14: LEDS = ~0x22;
                  break;
        case 15: LEDS = ~0x44;
                  break;
        case 16: LEDS = ~0x88;
                  break;

        default: LEDS = 0xFF;
    }
    switch (key_value)
    {
        case 0xee: //1110 1110
                    return_value = 1;

```

```

    }
}
}
/*****
矩阵键盘子函数，
通过判断按下的不同的按键返回不同的值
*****/
uchar KeyScan()
{
    uchar row,list,key_value, return_value;

    KEYS=0x0f;
//初始状态,行号 (P3^0~P3^3) 高电平,列号
(P3^4~P3^7) 低电平
    if(KEYS != 0x0f)
    {
        Delay_Ms(20);
//按键消抖动 (延时实现)
        if(Delay_Ms != 0x0f)
//按键被按下, 初始状态改变
        {
            row = KEYS;
//读取矩阵键盘的行号
        }
        KEYS = 0xf0;
//初始状态反转
        /*****
        注意程序运行到这里时按键仍是
        被按下的
        *****/
        // 1110 0000 | 0000 1110 ==1110 1110
        list = KEYS; //
        读取矩阵键盘的列号
        key_value = row | list;
        //按位'或', 通过行号, 列号唯一确定矩
        阵按键值

        break;
        case 0xde: //1101 1110
                    return_value = 2;
        break;

```

<pre> case 0xbe: //1011 1110 return_value = 3; break; case 0x7e: //0111 1110 return_value = 4; break; case 0xed: //1110 1101 return_value = 5; break; case 0xdd: //1101 1101 return_value = 6; break; case 0xbd: //1011 1101 return_value = 7; break; case 0x7d: //0111 1101 return_value = 8; break; case 0xeb: //1110 1011 return_value = 9; break; case 0xdb: //1101 1011 return_value = 10; break; </pre>	<pre> case 0xbb: //1011 1011 return_value = 11; break; case 0x7b: //0111 1011 return_value = 12; break; case 0xe7: //1110 0111 return_value = 13; break; case 0xd7: //1101 0111 return_value = 14; break; case 0xb7: //1011 0111 return_value = 15; break; case 0x77: //0111 0111 return_value = 16; break; default : return_value = 0; break; } </pre>
---	---

```
KEYS = 0xf0;          //松手检测
```

```
while (KEYS != 0xf0); //当没有松手,将在此一直等待
}
```

```
return return_value; //返回读取的按键值
```

```
}
```

```
/******
```

```
延时函数
```

```
*****/
```

```
void Delay_Ms(uint tc) //函数延时 tc 毫秒
```

```
{
```

```
    uint i;
```

```
    while(tc != 0)
```

```
    {
```

```
        for(i = 0; i < 122; i++)
```

```
        //执行 122 次（经验值）空循环将耗
```

时 1 毫秒

```
        tc--;  
    }  
}
```

到此独立按键和矩阵键盘大家已经会使用了，可以将之前的 LED，数码管，蜂鸣器等结合起来，实现一些有趣的小玩意儿！

本章常见错误分析：

1.

```
Build target 'Target 1'  
compiling 独立按键.c...  
独立按键.C(9): error C141: syntax error near '}'  
Target not created
```

在“}”符号出有错，双击此处回到程序

```
void delay(unsigned int a)//简单的延时  
{  
    for (;a>0;a--)  
}
```

我们可以轻易的发现 for 后的空循环没有写{}。
改正后如下

```
void delay(unsigned int a)//简单的延时  
{  
    for (;a>0;a--)  
    {  
    }  
}
```

2.

```
Build target 'Target 1'  
compiling 独立按键.c...  
独立按键.C(34): error C141: syntax error near '.7'  
Target not created
```

在.7 处有错，双击回到程序

```
P0.7=0x00;
```

修改正确后:

```
PO=0x00;
```

```
Build target 'Target 1'
compiling anjian.c...
ANJIAN.C(79): warning C206: 'Delay_Ms': missing function-prototype
ANJIAN.C(79): error C267: 'Delay_Ms': requires ANSI-style prototype
ANJIAN.C(160): error C231: 'Delay_Ms': redefinition
ANJIAN.C(168): error C231: 'Delay_Ms': redefinition
Target not created
```

找到函数声明区对函数进行修改如下

```
void Delay_Ms(uint tc); //函数延时tc毫秒
```

第 48 页

```

Build target 'Target 1'
compiling anjian.c...
ANJIAN.C(12): error C141: syntax error near 'P0', expected '='
ANJIAN.C(31): error C202: 'LEDS': undefined identifier
ANJIAN.C(33): error C202: 'LEDS': undefined identifier
ANJIAN.C(35): error C202: 'LEDS': undefined identifier
ANJIAN.C(37): error C202: 'LEDS': undefined identifier
ANJIAN.C(39): error C202: 'LEDS': undefined identifier
ANJIAN.C(41): error C202: 'LEDS': undefined identifier
ANJIAN.C(43): error C202: 'LEDS': undefined identifier
ANJIAN.C(45): error C202: 'LEDS': undefined identifier

```

在 p0 处有错，双击回到程序

```

#define uint unsigned int
#define uchar unsigned char
#define KEYS P3 //假设P3是矩阵键盘
sbit LEDES P0
/*****

```

初学者很容易犯下这样的错误，在这里特别的提醒大家注意。Sbit 只能定义位不能定义整个 P0 口，Px 口只能通过#define 来进行宏定义。

改正后如下：

```

/*****
/*宏定义
/*管脚定义
*****/
#define uint unsigned int
#define uchar unsigned char
#define KEYS P3 //假设P3是矩阵键盘
#define LEDES P0
/*****
函数声明区

```

5.

```

Build target 'Target 1'
assembling STARTUP.A51...
compiling anjian.c...
ANJIAN.C(28): error C267: 'KeyScan': requires ANSI-style prototype
Target not created

```

在 KeyScan 处有错，双击回到程序

主函数
通过调用矩阵键盘子函数的返回值来点
亮不同的灯

```

/*****/
int main()
{
    uint a;
    while(1)
    {
        a = KeyScan(0);
        switch(a)
        {

```

我们并没有发现这条语句有语法上的错误，于是我们找到 KeyScan 子函数

```

/*****
矩阵键盘子函数，
通过判断按下的不同的按键返回不同的值
*****/
uchar KeyScan(void)
{
    uchar row,list,key_value, return_value;

    KEYS=0x0f; //初始状态
    if(KEYS != 0x0f)
    {
        Delay_Ms(20); //延时
        if(Delay_Ms(20) != 0x0f)

```

发现传入的参数为空，而上图中的参数为 0，非法。

修改后如下：

```

主函数
通过调用矩阵键盘子函数的返回值来点
亮不同的灯
/*****/
int main()
{
    uint a;
    while(1)
    {
        a = KeyScan();

```

反思与改错

工程四：中断系统及定时器/计数器

引例：单片机的“一心二用”

什么是中断，我们从一个生活中的例程引入。你正在家中看书，突然电话铃响了，你放下书本，去接电话，和来电话的人交谈，然后放下电话，回来继续看你的书。这就是生活中的“中断”的现象。所谓中断，就是指正常的工作过程被外部的事件打断了。

对于中断，我们要清楚地明白：

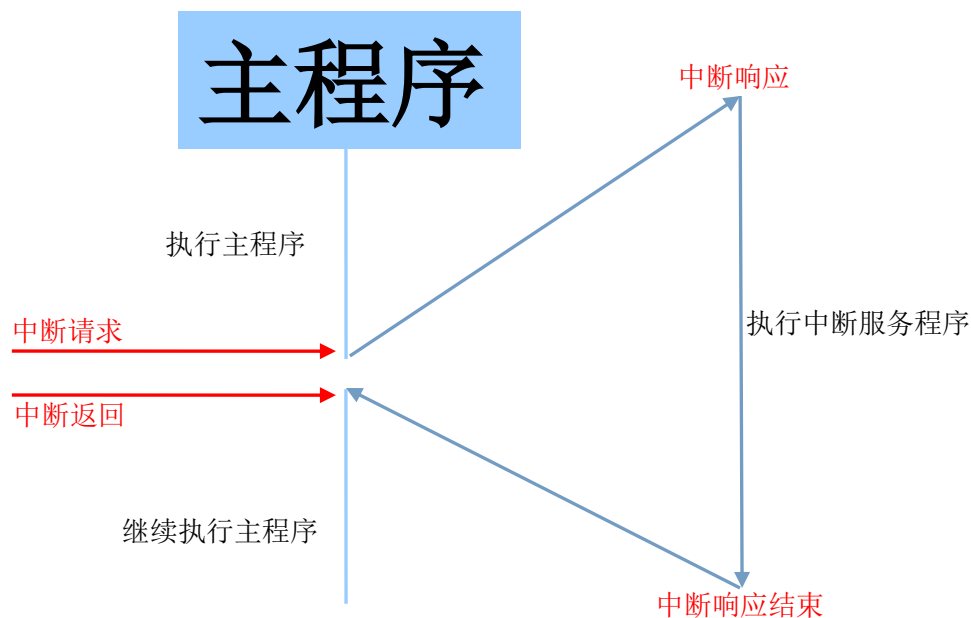
第一、什么可以引起中断，生活中很多事件能引起中断：有人按了门铃了，电话铃响了，你的闹钟闹响了，你烧的水开了等等事件，我们把能引起中断的事件称之为中断源，单片机中也有一些能引起中断的事件。

第二、中断的嵌套与优先级处理：设想一下，我们正在看书，电话铃响了，同时又有人按了门铃，你该先做那样呢？所以这里存在一个优先级的问題，单片机中也是如此，也有优先级的问題。优先级的问題不仅仅发生在两个中断同时产生的情况，也发生在一个中断已产生，又有一个中断产生的情况。比如你正接电话，有人按门铃的情况，或你正开门与人交谈，又有电话响了情况。考虑一下我们会怎么办吧。

第三、中断的响应过程：当有事件产生，进入中断之前我们必须先记住现在看书的第几页了，或拿一个书签放在当前页的位置，然后去处理不一样的事情（因为处理完了，我们还要回来继续看书）：电话铃响我们要到放电话的地方去，门铃响我们要到门那边去，也说的不一样的中断，我们要在不一样的地点处理，而这个地点常常还是固定的。

4.1 中断（服务图解及 8051 结构图）

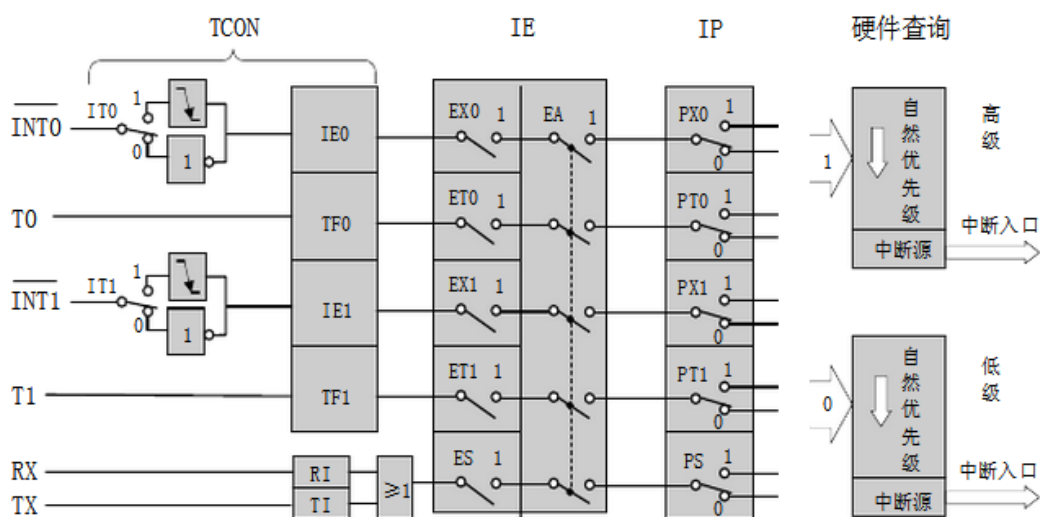
4.1.1 中断执行的程序顺序图以及解释：



解释:

当单片机主程序(main 函数)在执行的时候,由外部中断或者定时器/计数器触发中断请求。
 (内部中断记数或定时溢出时即触发中断请求)此时中断控制器向 CPU 发送中断请求,并由 CPU 选择(中断存在优先级)执行中断服务程序并且暂停中断发生时刻的主程序。当中断服务程序执行完毕后,继续回到中断发生时暂停的主程序位置,继续执行主程序。

4.1.2 80C51 中断系统的结构图:



解释:

80C51 的中断系统有 5 个中断源(8052 有 6 个),2 个优先级,可实现二级中断嵌套。

由上图可得出 5 个结论:

- 1、(P3.2)可由 IT0(TCON.0)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.2 引脚上出现有效的中断信号时,中断标志 IE0(TCON.1)置 1,向 CPU 申请中断。
- 2、(P3.3)可由 IT1(TCON.2)选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.3 引脚

上出现有效的中断信号时，中断标志 IE1(TCON.3)置 1,向 CPU 申请中断。

3、TF0 (TCON.5)，片内定时/计数器 T0 溢出中断请求标志。当定时/计数器 T0 发生溢出时，置位 TF0，并向 CPU 申请中断。

4、TF1 (TCON.7)，片内定时/计数器 T1 溢出中断请求标志。当定时/计数器 T1 发生溢出时，置位 TF1，并向 CPU 申请中断。

5、RI (SCON.0) 或 TI (SCON.1)，串行口中断请求标志。当串行口接收完一串行数据时置位 RI 或当串行口发送完一帧串行数据时置位 TI，向 CPU 申请中断。

4.2 定时器计时器原理

4.2.1 定时器计时器内部原理：

定时器/计数器的核心部件就是一个加法（也有是减法）的计数器，其本质是对脉冲进行计数。只是计数的脉冲来源不同；如果计数的脉冲来自系统时钟，则为定时方式，此时定时器每个机器周期得到一个计数脉冲，计数加 1；如果计数脉冲来自单片机外部引脚(T0 为 P3.4，T1 为 P3.5)，则为计数方式，每来一个脉冲加 1。

4.2.2 定时器定时功能使用原理：

定时器工作前先装入初值，利用软件将初值装入 TH0 和 TL0 或 TH1 和 TL1，高位数装入 TH0 和 TH1，底位数装入 TL0 和 TL1。当发出启动命令后，装初值寄存器开始计数，每一个机器周期（10 纳秒（ns）为一个时钟周期）加一次，加到满值（各位全为 1）。若再加 1，则会溢出，同时将初值寄存器清零。如果继续计数定时，则需要重新赋初值。

这种计时模式就如同古人使用沙漏计时，比如流过 1000 粒沙使用的时间是一秒，那么给沙漏装 10000 粒沙漏完的时间就是 10 秒。而给定时器装初值就如同给接沙的容器里先放一些沙，到这个容器装满时漏下的的沙就是容器的容量减去开始放的沙。用这种方式就可得出装满的时间是可根据初值确定的，同时也是可以计算的。

4.3 中断

4.3.1 中断发生的条件

对于计时器，当寄存器 TH，TL 装满（全为 1），再加 1 就装不下就溢出，则进入中断，这时有个标志 TF = 1。

对于外部中断，根据你选择的中断方式来确定中断条件，比如选择上升沿方式，就是外部接口输入上升沿（低电平到高电平），就会发生中断。

4.3.2 中断寄存器介绍

中断的 SFR

位	7	6	5	4	3	2	1	0
符号	EA	EC	ET2	ES	ET1	EX1	ET0	EX0

中断使能控制寄存器IE

EA:总中断使能位，总阀门。

EC:PCA中断使能位
ET2:定时器2中断使能位
ES:串口中断

ET1:定时器1溢出中断使能
EX1:外部中断1使能
ET0:定时器0溢出中断使能
EX0:外部中断0使能
注：使能就是开关的意思。

特殊功能寄存器TCON

位	7	6	5	4	3	2	1	0
符号	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TF：定时器溢出标志。溢出时，该位自动置 1。中断执行时硬件清零，或者软件清零。
TR：定时器运行控制位，置 1 开始计时，清 0 停止计时。

定时器的特殊功能寄存器 TMOD

位	7	6	5	4	3	2	1	0
符号	T1 GAT	T1 C/T	T1 M1	T1 M0	T0 GAT	T0 C/T	T0 M1	T0 M0

T1 和 T0 分别代表单片机两个计数器
GATE:该位被置位时为门控位。仅当 TR1 被置位并且 INT1 脚为高，定时器开始计数。当该位被清零时，只要 TR1 被置位，定时器 1 马上开始计数。
C/T:该位为 0 的时候，用作定时器，该位为 1 的时候，用做计数器。

M1,M0

00	01	10	11
模式 0	模式 1	模式 2	模式 3

模式 1：16 位的计数器。(TH1,TL1)
模式 2：自动装载 8 位计数器。主要应用在串口波特率发生器。
模式 0&模式 3：几乎不用。
TCON 和 TMOD 复位后都会自动变成 0x00.
 $12 * (65536 - x) / 11059200 = 0.001$

4.3.3 使用定时器的方法

第一：设置特殊功能寄存器 **TMOD**，使之工作在需求的状态。
 第二：设置计数寄存器的初值，精确设定好定时时间。
 第三：设置特殊功能寄存器 **TCON**，通过打开 **TR** 来让定时器进行工作。(也可设置为中断模式)

TH1 = 0xfc; TL1 = 0x66;
 $12(65536 - x) / 11059200 = 0.02s$

4.3.4 中断配置示例

void Time_start(void)

```
{
    TMOD &= 0x0f;      //设置定时器模式
    TMOD |= 0x20;      //使用或且逻辑来避免对 TMOD 中的另一个定时器
    误操作
    TH1 = 0xB0;        //定时器初始值
    TL1 = 0x3C;

    TR1 = 1;           //开启定时器 1
    EA = 1;             //开启总中断
    ET1 = 1;           //开启定时器 1 中断允许位
    TF1 = 0;           //将定时器中断溢出位置为 0（表示未溢出）
}
```

void Time_t(void) interrupt 3 //前面函数名 void Time_t(void)是自己取名字
 interrupt 表示该函数是中断固定写法 3 表示是定时器 1 中断的函数

```
{
    static int d;      //函数内容是根据你写的程序来写
    if (d == 4)
    {
        d = 0;
    }
    d++;
}
```

知识小节

中断类型	中断序号
外部中断 0	0
定时器 0	1
外部中断 1	2
定时器 1	3
串口中断	4

4. 4 用中断来做一个时钟（进一步了解中断）

```
#include<reg52.h>
#define uchar unsigned char
#define uint unsigned int

#define duan P1 //与下面四句 SBIT 联合起来使用数码管
sbit wei1=P2^0;
sbit wei2=P2^1;
sbit wei3=P2^2;
sbit wei4=P2^3;

uchar shu,fen,miao;
uchar code table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90}; //数码管 0 到 9 码表
void delay(int z); //毫秒级延时函数
void init();
void display(uchar fen, uchar miao);
void main(void)
{
    init();//在主函数里面初始化定时器设置
    while(1)
    {
        display(fen,miao);
    }
}
void init(void) //初始化
{
    shu=0;
    miao=59;
    fen=0;
    TMOD=0x01; //设置开启定时器 0
    TH0=(65536-50000)/256; //对定时器高八位赋初值 每 0.05 秒进一次中断
    TL0=(65536-50000)%256;

    EA=1; //总开关
    ET0=1;
    TR0=1;
}
void timer0() interrupt 1 //定时器 0 的中断
{
    TH0=(65536-50000)/256; //再次赋初值，为下次进中断做准备
    TL0=(65536-50000)%256;
    shu++; //设置一个标志数
```

```

    if (shu == 20)          //当 shu==20 的时候，表示经过了 1 秒钟时间，来执行下面的命令
    {
        shu = 0;           //把标志数清零，一边下次计数
        miao++;            //秒数增加 1
        if(miao == 60)
        {
            miao = 0;
            fen++;
            if (fen == 60)
            {
                TR0=0;      //当达到 60 分时，关闭定时器 0
            }
        }
    }

}

void display(uchar fen, uchar miao)    //显示时间函数
{
    duan=table[miao%10];
    wei1=0;
    delay(1);
    wei1=1;

    duan=table[miao/10];
    wei2=0;
    delay(1);
    wei2=1;

    duan=table[fen%10];
    wei3=0;
    delay(1);
    wei3=1;

    duan=table[fen/10];
    wei4=0;
    delay(1);
    wei4=1;
}

void delay(int z)
{
    int i, j;

    for (i=0; i<z; i++)

```

```

        for (j=0; j<110; j++)
            ;
    }

```

4.5 中断即时应用——>更快的掌握中断

接下来我们来写一个可调的分秒时钟

//秒表：s17 开始，s18 暂停，s19 继续，s20 复位

```
#include<reg52.h>
```

```
//位定义
```

```
sbit rst=P3^7;
```

```
sbit temp=P3^6;
```

```
sbit key=P3^5;
```

```
sbit start=P3^4;
```

```
//数据处理
```

```
unsigned char code Tab[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90}; //0
```

到 9 码表

```
unsigned char code Tab1[]={0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10}; //
```

带小数点的码表

```
unsigned char int_time, second, minute; //全局变量
```

```
unsigned char count=0;
```

```
void delay(void) //延时函数
```

```
{
```

```
    unsigned char j;
```

```
    for(j=0;j<125;j++);
```

```
}
```

```
//显示秒子函数
```

```
void DisplaySecond(unsigned char s)
```

```
{
```

```
    P1=Tab[s/10]; //显示十位
```

```
    P2=0xfd; //P2.1 引脚输出低电平,第二个数码管亮
```

```
    delay(); //延时
```

```
    P2=0xff; //P2 口全为高电平 所有数码管熄
```

```
    P1=Tab[s%10]; //显示个位
```

```
    P2=0xfe; //P2.0 引脚输出低电平, 第一个数码管点亮
```

```
    delay();
```

```
    P2=0xff;
```

```
    //从上到下是一个动态闪烁过程
```

```
}
```

```
//显示分子函数
```

```
void DisplayMinute(unsigned char m)
```

```
{
```

```
    P1=Tab1[m%10]; //显示个位
```

```

P2=0xfb; // P2.3 引脚输出低电平,第三个数码管点亮
delay();
P2=0xff;

P1=Tab[m/10]; //显示十位
P2=0xf7; // P2.4 引脚输出低电平, DS4 点亮
delay();
P2=0xff;
}
void main(void)
{

TMOD=0x01; //使用定时器 T0
EA=1; //开中断总允许
TH0=(65536-50000)/256; //定时器高八位赋初值
TL0=(65536-50000)%256; //定时器低八位赋初值
TR0=1; //启动定时器 0
EX0=1; //开外部中断 0
IT0 = 1; // 低电平触发（1 为下降沿触发）（0 为低电平触发详细了解看数
据手册）
int_time=0; //中断计数变量初始化
second=0; //秒计数变量初始化
minute=0; //分钟计数变量初始化
while(1)
{
    if (start==0)//按键 P3^4 判断
    {
        ET0=1; ////打开定时器 0 中断
        while(1)
        {
            DisplaySecond(second); //调用秒显示子程序
            delay();
            DisplayMinute(minute); //调用分钟显示子程序
            delay();
            if (key==0)//按键 P3^5 判断
            {
                P3=0xfb; //或 P3^2=0 此 I/O 口对应是外部中断 INT0 如果按
                键按下 此时进入外部中断
            }
            if(rst==0)//按下 P3^7 打开复位
            {
                ET0=0; //关闭定时器 0
                int_time=0;
                second=0;
            }
        }
    }
}

```

```

        minute=0;
        while(start!=0)
        {
            P2=0xf0;
            P1=0xc0;
        }
        ET0=1;
    }
}
else//如果复位以后没有按键按下
{
    P2=0xf0; //四个数码管全开
    P1=0xc0; //显示 0000
}
}
}

void interserve() interrupt 1 //定时器 0 中断
{
    TH0=(65536-50000)/256; //定时器重新赋初值
    TL0=(65536-50000)%256;
    int_time++;

    if(int_time==20)
    {
        int_time=0; //中断计数变量清 0
        second++; //秒计数变量加 1
    }
    if(second==60)
    {
        second=0; //如果秒计满 60，将秒计数变量清 0
        minute++; //分钟计数变量加 1
    }
}

void INT_0() interrupt 0 //外部中断
{
    count++;
    delay();
    if (count%2!=0)
    {
        P0=0x00; //点亮 LED
        while(1)
        {

```

```

        DisplaySecond(second); //调用秒显示子程序
        delay();
        DisplayMinute(minute); //调用分钟显示子程序
        delay();
        if (temp == 0) //P3^6 按键判断
        {
            count++;
            P0=0xff;    //P0 口全为高电平
            P3=0xff;    //P3 口全为高电平
            break; //结束外部中断程序
        }
    }
}
}
}

```

此程序所有定时器 0 中断和外部 IN0 中断的信息都可以从数据手册查到，所以不必需要去死记硬背，只需要懂怎么去看数据手册你就算学会了很多芯片了，学习 51 单片机，只是引领的过程，到以后我们需要用更多的芯片，怎么办？没人再来教你，学会看懂数据手册，让你从根本上理解 51 芯片。

4.6 感悟

中断和定时器/计数器这一节比较难懂，当初学长在学这一节的时候也是花了好长的时间。有两点给大家的建议：第一，自己多多写程序，不管是把别人的程序 copy 过来还是自己一点一点的琢磨，这个过程对理解中断和定时器/计数器有很大的帮助；第二，在这节的时候，大家开始接触芯片的数据手册，数据手册对写程序的人来说可谓是笑傲江湖，可见其重要性，希望大家现在开始学着看一点英文版的 PDF。

4.7 错误是最好的老师

中断中的常见错误：

1) 关键字出错

```

void Time_start(void)
{

    TMOD &= 0x0f;    //设置定时器模式
    TMOD |= 0x20;
    TH1 = 0xB0;
    TL1 = 0x3C;

    TR1 = 1;
    EA = 1;
}

```

```

    ET1 = 1;
    TF1 = 0;

}

```



错误现象:

compiling 模拟电梯.c...

模拟电梯.c(221): error C202: 'TH': undefined identifier

模拟电梯.c(222): error C202: 'TL': undefined identifier

模拟电梯.c - 2 Error(s), 0 Warning(s).

错误分析:

TH, TL 在第 221, 222 行, 没有定义 (undefined identifier)

因为 TH1, TL1, TH0, TL0, 是被包括在头文件, <reg51.h> 中, 所以, 它是关键字, 是被定义了的。

而 TH, TL 并未被定义包含到头文件中, 所以, 出现错误。

错误更改:

将 TH, TL, 改为 TH1, TL1。

因为后面中断开启允许位是 ET1, 所以这里是用的定时器 1, 而不是 0, 所以改为 TH1 而不是 TH0。

2) 定时器初始化出现覆盖错误

void Time_start(void)

```

{

    //设置定时器模式
    TMOD = 0x20;
    TMOD = 0x02;
    TH1 = 0xB0;
    TL1 = 0x3C;

    TH0 = 0xc0;
    TL0 = 0xb0;

    TR1 = 1;
    EA = 1;
    ET1 = 1;
    TF1 = 0;
}

```

```

    TR0 = 1;
    ET0 = 1;
    TF0 = 0;
}

```

程序解释：

本程序想分别使用两个定时器，一个定时为 TH1 = 0xB0; TL1 = 0x3C; 一个定时为 TH0 = 0xc0; TL0 = 0xb0; 定时时间不一样，分别在 2 模式下，即

TMOD 寄存器

位	7	6	5	4	3	2	1	0
符号	T1 GAT E	T1 C/T	T1 M1	T1 M0	T0 GAT E	T0 C/T	T0 M1	T0 M0

高四位为 0010，为定时器 1 的控制

低四位为 0010，为定时器 0 的控制

错误现象：

无警告，无错误，但是程序实际运行错误。

错误分析：

TMOD 寄存器本来想高四位为 0010，为定时器 1 的控制，低四位为 0010，为定时器 0 的控制，但是这里采用的是 8 位整体赋值。

为 TMOD = 0x20; TMOD = 0x02;

第一次把高四位变成 0010，第四位为 0000;

第二次把高四位变成 0000，第四位为 0010;

这样，第一次赋值就毫无意义，但是却是初学者常常犯得错误。理所当然的认为一次控制高四位，即定时器 1，一次控制低四位，即定时器 0，认为两次是没有关联的。

错误更改：

通过与 (&)，或 (|) 逻辑运算符号来控制即可使两次互相不影响。

定时器 1 的设置：

TMOD &= 0x0f; // 通过与 (&) 0 有效 (0 有效是指遇 0 变 0)，将高四位初始化全
// 部置 0，而低四位全部为 1，是无效的，不会影响低四位原有
// 的值，从而达到了保存的目的

TMOD |= 0x20; // 通过或 (|) 1 有效，将高四位变为 0010；而低四位全部为 0，
// 在或运算中是无效的，不会影响低四位原有的值，从而达到了
// 保存的目的

定时器 0 的设置同理

TMOD &= 0xf0;

TMOD |= 0x02;

3) 中断程序连接序号出错

```
void Time_t(void) interrupt 3
{
    static int d;
    if (d == 4)
    {
        d = 0;
    }
    segdisplay(d);
    d++;
}
```

程序解释:

本程序想让定时器 0 产生中断后进入该中断程序。

错误现象:

无警告，无错误，但是程序实际运行错误。不能运行中断当中的程序。

错误分析:

如果能让定时器 0 产生的溢出进入中断，本应该是连接序号为 1，但是这里的连接序号是 3，是定时器 1 产生的中断的中断程序，所以定时器 0，产生的中断当然不能进入中断程序。

错误更改:

将中断程序名 void Time_t(void) interrupt 3

改为 void Time_t(void) interrupt 1 即可，则是定时器 0 的中断程序了。

错误中的知识小结:

中断类型	中断序号
外部中断 0	0
定时器 0	1
外部中断 1	2
定时器 1	3
串口中断	4

4.8 几种中断的模板

1) 定时器中断

Step 1: 初始化定时器

```
void Time_start(void)
```

```

{

    TMOD &= 0x0f;    //设置定时器模式
    TMOD |= 0x20;
    TH1 = 0xB0;
    TL1 = 0x3C;

    TR1 = 1;
    EA = 1;
    ET1 = 1;
    TF1 = 0;

}

```

要求：1 选择模式 TMOD 要选择定时器几
 2 选择模式 TMOD 要选择定时器的定时方式
 3 初始化定时器的初始寄存器 TH, TL
 4 打开定时器和定时器中断

Step 2: 中断内容

```

void Time_t(void) interrupt 3
{
    static int d;
    if (d == 4)
    {
        d = 0;
    }
    segdisplay(d);
    d++;
}

```

要求：1. interrupt 数字一定要加上。
 2. 中断内容不能使死循环。

2) 计数器中断

Step 1: 初始化计数器

```

oid Time_start(void)
{

    TMOD &= 0x0f;    //设置定时器模式

```

```

    TMOD |= 0xd0;
    TH1 = 0xB0;
    TL1 = 0x3C;

    TR1 = 1;
    EA = 1;
    EX1 = 1;
    TF1 = 0;

}

```

注意：注意计数器和定时器在初始化上的区别。仅仅是 TMOD 选择了计数器，和中断打开的是 EX 计数器中断。

注意计数器的中断时要在外部脉冲（电流信号）输入来刺激中断。

Step 2: 中断内容

```

void Time_t(void) interrupt 2
{
    static int d;
    if (d == 4)
    {
        d = 0;
    }
    segdisplay(d);
    d++;
}

```

注意：中断内容基本相同，就是中断序号改变成 2 了。

3) 外部中断（串口中断）

Step 1: 外部中断配置（打开外部中断）

```

ES = 1;
EA = 1;

```

Step 2: 中断内容

```

void Time_t(void) interrupt 4
{
    static int d;
    if (d == 4)

```

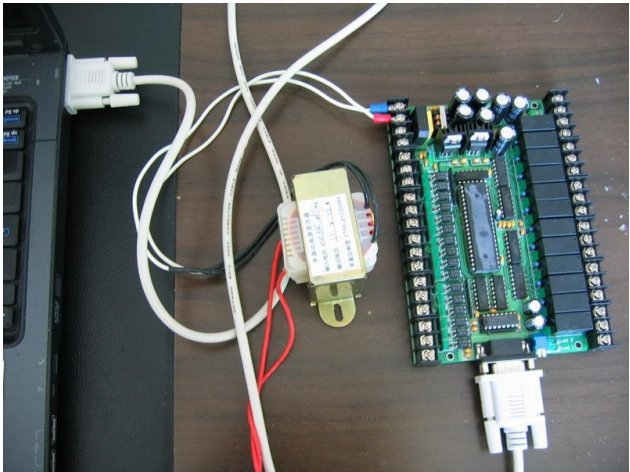
```
{  
    d = 0;  
}  
segdisplay(d);  
d++;  
}
```

反思与改错

工程五：串口通信及串口中断

导引：芯与芯的桥梁——串口

我们已经学习了单片机的基本功能，但是在实际生活中，每块单片机并不是独立自主的运作的，就像我们人类社会，我们不能独自脱离社会群体生活一样，人与人之间需要沟通，需要交流，这样才能构建一个信息网络，社会功能才会更加完善。而单片机系统也如此，单片机（下位机）要实现一些大的功能或组成大的系统时，需要多个单片机系统模块相互配合，相互合作，而最终结果则更是需要返回到 PC 机或者其他上位机间接反映给实际操作人员，从而达到便于控制，集中管理，快速高效的效果。



这其中所涉及到的最基本的通信原理即串口通信技术，那我们要怎样才能实现下位机与下位机或者下位机与上位机之间的串口通信呢？

在本章将详细介绍单片机的基本串口通信技术怎样由 C 语言程序实现的方法……

5.1 串口

5.1.1 串口概念：

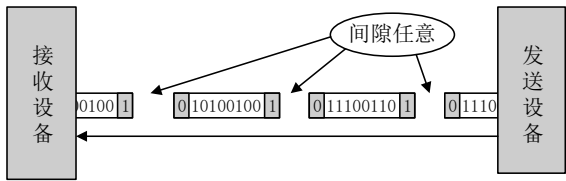
串行通信是将数据字节分成一位一位的形式在一条传输线上逐个地传送。（如图 1 示）

5.1.2 串口特点：

传输线少，长距离传送时成本低，且可以利用电话网等现成的设备，但数据的传送控制比并行通信复杂。



（图 1）



（图 2）

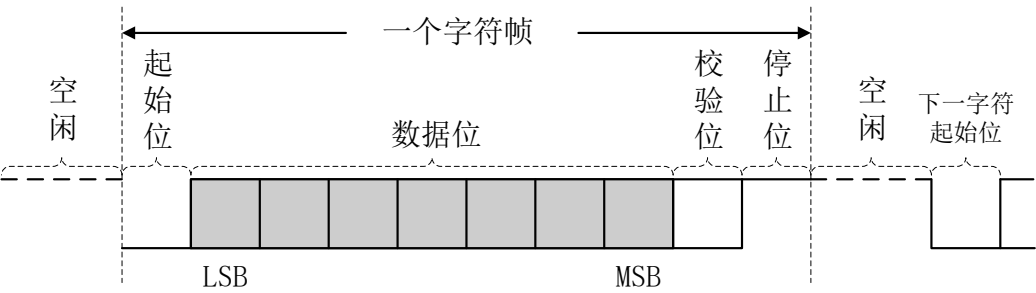
5.2 异步通信与同步通信

5.2.1 异步通信

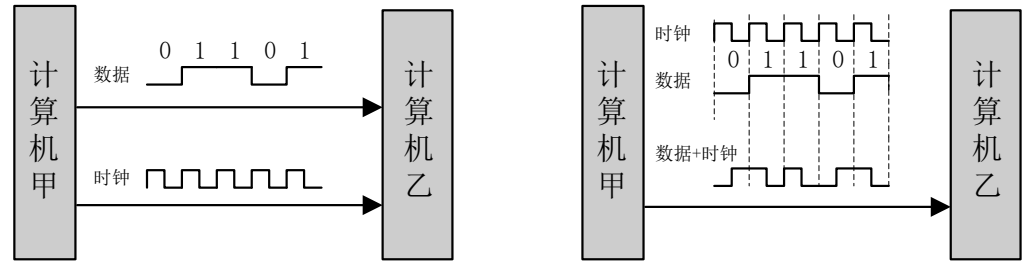
异步通信是指通信的发送与接收设备使用各自的时钟控制数据的发送和接收过程。为使双方的收发协调，要求发送和接收设备的时钟尽可能一致。（如图 2 示）

异步通信是以字符（构成的帧）为单位进行传输，字符与字符之间的间隙（时间间隔）是任意的，但每个字符中的各位是以固定的时间传送的，即字符之间不一定有“位间隔”的整数倍的关系，但同一字符内的各位之间的距离均为“位间隔”的整数倍。

异步通信的数据格式：



(图 3)



(图 4)

异步通信的特点：不要求收发双方时钟的严格一致，实现容易，设备开销较小，但每个字符要附加 2~3 位用于起止位，各帧之间还有间隔，因此传输效率不高。

5.2.2 同步通信

同步通信时要建立发送方时钟对接收方时钟的直接控制，使双方达到完全同步。此时，传输数据的位之间的距离均为“位间隔”的整数倍，同时传送的字符间不留间隙，即保持位同步关系，也保持字符同步关系。发送方对接收方的同步可以通过两种方法实现。（如图 4 示）

5.3 串行通信的传输方向

5.3. 1 单工

(如图 5 示)

单工是指数据传输仅能沿一个方向，不能实现反向传输。

5.3. 2 半双工

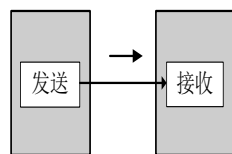
(如图 6 示)

半双工是指数据传输可以沿两个方向，但需要分时进行。

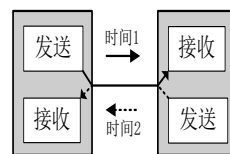
5.3. 3 全双工

(如图 7 示)

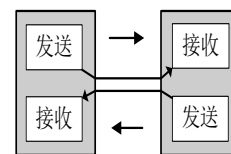
全双工是指数据可以同时进行双向传输。



(图 5)

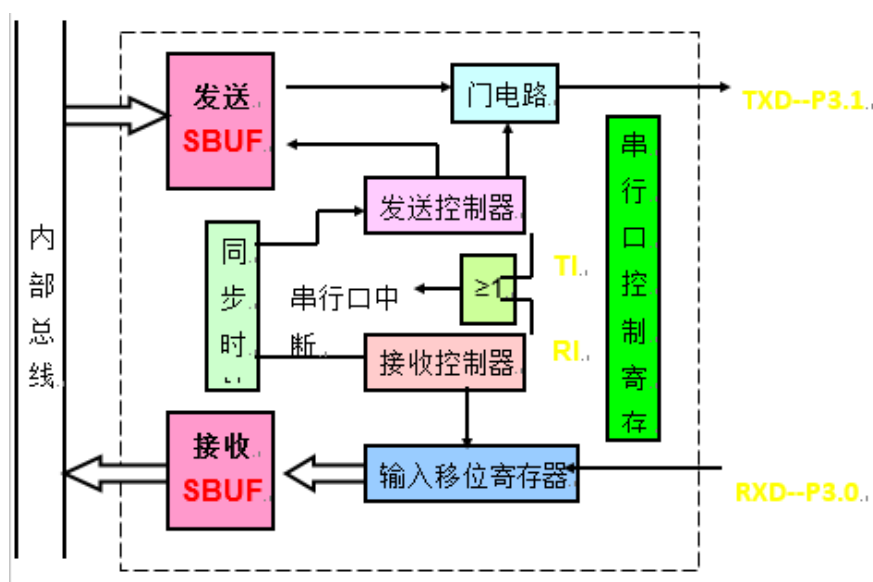


(图 6)



(图 7)

5.4 串行口缓冲寄存器 SBUF



SBUF 是串行口缓冲寄存器，包括发送寄存器和接收寄存器。

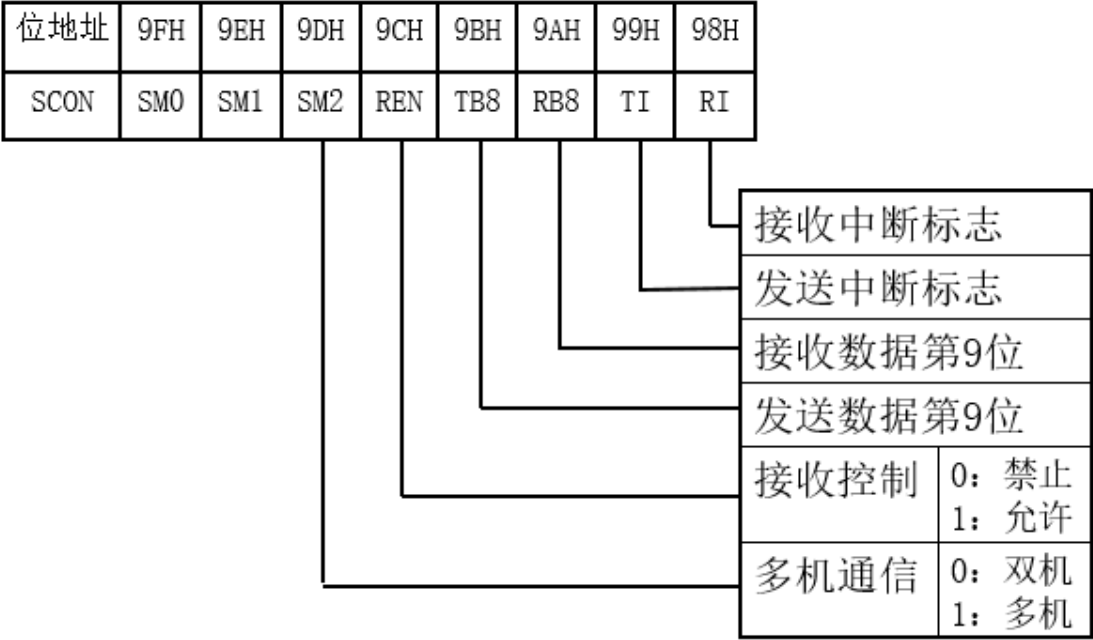
注意：当 CPU 向 SBUF 写数据时，SBUF 就是发送寄存器。当 CPU 读取 SBUF 的数据时，SBUF 就是接收寄存器。因为每一个单片机只有一个 SBUF。

示例： SBUF = 'A'; //发送字符'A'

Temp=SBUF; //接收 SBUF 的值存到 Temp 里

5.5 串行控制寄存器 SCON

串行控制寄存器 SCON 用于设置串行口的工作方式、监视串行口的工作状态、控制发送与接收的状态等。它是一个既可以字节寻址又可以位寻址的 8 位特殊功能寄存器。



(1) SM0 SM1: 串行口工作方式选择位。其状态组合所对应的工作方式下表所示。

SM0	SM1	工作方式	功 能 说 明
0	0	0	同步移位寄存器输入/输出，波特率固定为fosc/12
0	1	1	10位异步收发，波特率可变(T1溢出率/n, n=32或16)
1	0	2	11位异步收发，波特率固定为f0sc/n, n=64或32)
1	1	3	11位异步收发，波特率可变(T1溢出率/n, n=32或16)

(2) REN: 串行接受允许控制位。该位由软件置位或复位。当 REN=1，允许接收；当 REN=0，禁止接收。

(3) TI: 发送中断标志位。TI=1，表示已结束一帧数据发送，可由软件查询 TI 位标志，也可以向 CPU 申请中断。

(4) RI: 接收中断标志位。RI=1，表示一帧数据接收结束。可由软件查询 RI 位标志，也可以向 CPU 申请中断。

注意：RI 在任何工作方式下也都必须由软件清 0。

5.6 波特率

5.6.1 电源控制寄存器 PCON

PCON 是一个特殊的寄存器，除了最高位 SMOD 外，其他位都是虚设的。

PCON	D7	D6	D5	D4	D3	D2	D1	D0
名称	SMOD	---	---	---	GF1	GF0	PD	IDL

SMOD: 串行口波特率倍增位。在工作方式 1~工作方式 3 时，若 SMOD=1，则串行口波特率增加一倍。若 SMOD=0，波特率不加倍。系统复位时，SMOD=0。

通信双方要求数据传输的速率相同，这个速率就叫做波特率。

用定时器 T1 作为波特率发生器（关于定时器，可以查阅工程三）：

定时/计数器工作方式设置表		
M1M0	工作方式	说明
00	方式 0	13 位定时/计数器
01	方式 1	16 位定时/计数器
10	方式 2	8 位自动重装定时/计数器
11	方式 3	T0 分成两个独立的 8 位定时/计数器；T1 此方式停止计数

设置 T1 步骤：

- 1、TMOD = 0x20; // 定时器 1，方式 2，8-bit 重装
- 2、TH1 = 0xF3; // 设置 2400 波特率 晶振 12MHz
- 3、TR1 = 1; // 打开定时器 1

波特率的计算公式：

SMOD=0 时，波特率为晶振频率的 1/64；

SMOD=1 时，为晶振频率的 1/32。

$$\text{波特率} = (2^{\text{SMOD}} / 32) \times \text{定时器 1 的溢出率}$$

因此可以看出根据定时器的溢出率不同，而达到不同需求的波特率。但因为其计算麻烦，因此在使用时，一般由软件计算快速得出。

5.7 串口初始化

在讲完了所有初始化串口需要的知识后，现在来自行动手设置一个串口。

1. 初始化 SCON 寄存器（即 SCON = 0x50;）;
2. 设置 定时器 1 为重装模式（TMOD = 0x20;）;
3. 设置重装值，设置波特率为 2400（TH1 = 0xF3;）
4. 打开定时器 1（TR1 = 1;）

（以上步骤缺少一个，串口都无法正常初始化）

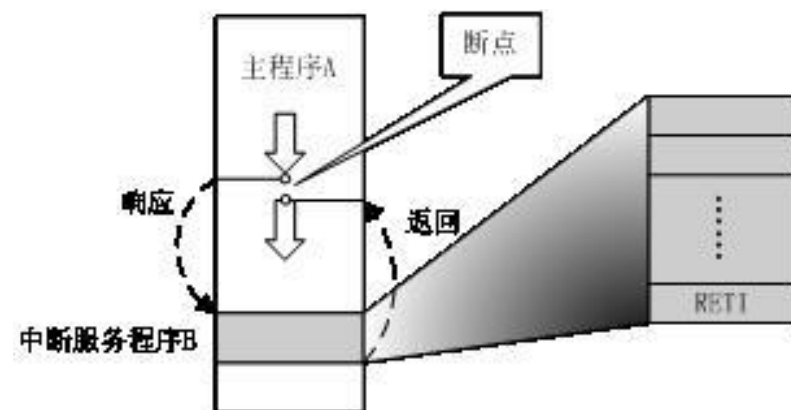
5.8 串口中断

经过上面的讲解，现在进入串口中断核心。

第一：

串口中断初始化

串行中断：单片机一旦接受到串行数据时，即 RI（SCON 的第一个地址，接受中断标志），被单片机硬件置为 1，单片机立即进入串行中断，同定时器中断一样，进入中断后执行函数里面程序，执行完后，返回进入中断时的位置。



串行中断就不像定时器那么复杂啦--->一步定乾坤

ES = 1; 打开串行中断

当然最后免不了要把中断的总开关打开

EA = 1; 打开中断总开关 （只要有中断服务程序，此语句一定存在）

第二：

中断函数中程序处理

void uart_ser(void) interrupt 4 //特别申明 如果次子函数在主函数后面 不用在主函数前面进行申明

```
{
    uchar temp; //建立一个无符号的字符变量
    if (RI) //RI=1 进入中断 判断是否是发送数据导致中断产生
    {
        RI = 0; //接受中断标志 RI = 1 表示接受一帧数据 需要软件清 0
        temp = SBUF; //读入缓冲区的值
        SBUF = temp; // 接受返回电脑
    }
    if (TI)
    {
        TI = 0; //发送数据结束标志 TI 需要软件清 0
    }
}
```

注意：当进入中断后 RI=1,每次需要手动清零（RI=0）

第三：

通信内容的子程序（注：没有此内容，也可以进行通信，当然那只能手动发送）

以下子程序具有发送字符串功能，不用手动发送，单片机打开串口自动发送

void send2(unsigned char dat)

```
{
    SBUF = dat;
```

```

        while(!TI);
        TI = 0;
    }
void send1(unsigned char *s)
{
    while(*s!='\0')
    {
        SendByte(*s);
        s++;
    }
}

```

5.9 让我们来写第一个完整的串口通信 C 程序

5.9.1 用串口通信向 PC 机发送字符串"Hello World!"

```

#include<reg52.h>                //包含头文件

void send1(unsigned char *s);
void delay(unsigned int t);
void send2(unsigned char dat);
void init (void)// 串口寄存器和中断初始化写在一起
{
    SCON  = 0x50;                // SCON: 模式 1, 8-bit UART, 使能接收
    TMOD  = 0x20;                // TMOD: timer 1, mode 2, 8-bit 重装
    TH1   = 0xF3;                // TH1: 重装值 2400 波特率 晶振 12MHz
    TR1   = 1;                  // TR1: timer 1 打开
}
void main (void)
{
    //连接好串口或者 usb 转串口至电脑，下载该程序，打开电源
    //打开串口调试程序，将波特率设置为 2400，无奇偶校验

    init();
    while (1)
    {
        // 主函数
        send1("Hello world!\n");
        delay(50000);
    }
}
void send2(unsigned char dat)
{
    SBUF = dat;
    while(TI == 0);              //发送标志位
    TI = 0;
}
void send1(unsigned char *s)

```

```

{
    while(*s!='\0')    //\0 表示字符串结束标志，通过检测是否字符串末尾
    {
        send2(*s);
        s++;
    }
}
void delay(unsigned int t)
{
    while(--t);        //串口缓冲作用
}

```

5.9.2 用串口通信接收信息控制 LED 灯之一

```

#include <REG52.H>
#define LED P0
unsigned char UART_buff;
bit New_rec = 0, Send_ed = 1, Money = 0;
//-----
void main (void)
{
    SCON = 0x50;    //串口方式 1, 8-n-1, 允许接收.
    TMOD = 0x20;    //T1 方式 2
    TH1 = 0xFD;     //9600bps@11.0592MHz
    TL1 = 0xFD;
    TR1 = 1;
    ES  = 1;        //开中断.
    EA  = 1;
    while(Money == 0);    //等着交费，呵呵，等着接收$.
    while(1) {
        if ((New_rec == 1) && (Send_ed == 1)) { //如果收到新数据及发送完毕
            SBUF = UART_buff; //那就发送.
            New_rec = 0;
            Send_ed = 0;
        }
    }
}
//-----
void ser_int (void) interrupt 4
{
    if(RI == 1) { //如果收到.
        RI = 0;    //清除标志.
        New_rec = 1;
        UART_buff = SBUF; //接收.
        if(UART_buff == '1') LED = 1;
    }
}

```

```

        if(UART_buff == '0') LED = 0;
        if(UART_buff == '$') Money = 1;
    }
    else {          //如果送毕.
        TI = 0;      //清除标志.
        Send_ed = 1;
    }
}

```

5.9.3 用串口通信接收信息控制 LED 灯之二

/*接受一个数组如果首为 0c 尾为 0f 则显示数组中间三个且反馈给计算机，否则显示 eeeeeee 且反馈给计算机*****/

<pre> #include<reg52.h> #define uchar unsigned char #define uint unsigned int uchar i,j,add; // uchar temp[15]; //控制显示数形 uchar temp1[5]; //暂存接收数据 uchar code position[]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb,0xfd, 0xfe}; //数码管位置数组 Uchar code figure[]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d, 0x07,0x7f,0x6f, 0x77,0x7c,0x39,0x5e,0x79,0x71,0x00}; //数形 void Init(); //初始化函数 void Check(); //核实接受的 数据 void Divide(); //将接收数据 进行分解 void Display(); //显示接收到 的数字 void Send(); //发送数据 void main() { Init(); while(1) { </pre>	<pre> if(add==4) { Send(); add=0; Check(); } } /*****初始化函数*****/ void Init() { TMOD=0x20; // 定时器 1 工作于 8 位自动重载模式, 用于产生波特 率 TH1=0xFD; // 波特率 9600 TL1=0xFD; SCON=0x50; // 设定串 行口工作方式 PCON&=0xef; // 波特率 不倍增 TR1=1; // 启动定 时器 1 EA=1; // 开总中 断 ES=1; // 串 行 中 断允许 P0=0x3f; P2=0x00; </pre>
---	--

```

        i=0;
        j=0;
        add=0;
    }

    /*****核实接收的数据*****/
    void Check()
    {
        Divide();
        if(temp1[0]==12&&temp1[4]==15)
        {
            while(!add)
            {Display();}
        }
        else
        {
            P2=0x00;
            P0=0x79;
        }
    }

    /***** 将接收数据进行分解
    *****/
    void Divide()
    {
        for(j=0;j<5;j++)
        {
            temp[3*j]=temp1[j]/16;
            temp[3*j+1]=temp1[j]%16;
            temp[3*j+2]=0x10;
        }
    }

    /***** 显示接收到的数字
    *****/
    void Display()

```

```

    {
        for(i=0;i<3;i++)
        {
            for(j=(3*i);j<(3*i+3)&&j<8;j++)
            {
                P0=0x00;
                P2=position[j];
                P0=figure[temp[j+3]];
            }
        }
    }

    /*****发送数据*****/
    void Send()
    {
        for(i=0;i<5;i++)
        {
            SBUF=temp1[i];
            while(!TI);
            TI=0;
        }
    }

    /*****中断服务函数*****/

    void serial () interrupt 4 using 3 //串口
    中断
    {
        temp1[add]=SBUF;
        if(RI)
        {
            RI=0;
            add++;
        }
    }
}

```

5.10 串口的拓展

串口的实际应用____调频呼吸灯

```
#include <reg52.h>
```

```
#include <stdio.h>
```

unsigned char Duanma[] = {0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90,0xc0}; //数码管显示呼吸灯频率，这里是数码管段码

int buff[4] = {1, 0, 0, 0}; //这里是数码管的显示缓存。注：缓存就是把要显示的数据全部先放到一个空间中，然后再把数据送到发送口的一个内存空间

sbit Led = P0^0;

int sbuf = 1; //串口发送缓存，这里使它默认为1，即1hz

<pre> void Time0_initial() //用于数码管扫描 的定时器0初始化 { TMOD &= 0xf0; TMOD = 0x02; TH0 = 0x06; TL0 = 0x06; EA = 1; ET0 = 1; TR0 = 1; } void Display(int n) //用于数码管显示的函数 { P2 = 0xef; P1 = Duanma[buff[n]]; switch(n) { case 0: P2 &= 0xfe; break; case 1: P2 &= 0xfd; break; case 2: P2 &= 0xfb; break; case 3: P2 &= 0xf7; break; default: P1 = 0xff; P2 = 0xff; </pre>	<pre> break; } } void InTime0 () interrupt 1 //定时器中断进行扫描数码管 { static int d; if (d == 4) { d = 0; } Display(d); d++; } void Delay_ms(int n)//延时函数 { int i; while (n--) { for (i = 0; i < i; i++); } } void BreathLed(int Hz)//呼吸灯函数，hz为呼吸频率 { int circle, i, j; circle = 100 / Hz;//100/hz就可以控制循环次数了 for (i = 0; i < circle; i++) //呼吸灯渐亮循环 { for (j = 0; j < 3; j++) { </pre>
--	---

<pre> Led = 0; Delay_ms(i); //亮的时间增加 Led = 1; Delay_ms(circle - i); //暗的时间减少，形成了渐亮的效果 } } for (i = 0; i < circle; i++) //呼吸灯渐灭循环 { for (j = 0; j < 3; j++) { Led = 1; Delay_ms(i); Led = 0; Delay_ms(circle - i); } } } void SBUF_initial() //串口初始化函数 { SCON=0X50; //设置为工作方式1 TMOD &= 0x0f; TMOD = 0x20; //设置计数器工作方式2 PCON=0X80; //波特率加倍 TH1=0XF3; //计数器初始值设置，注意波特率是4800的 </pre>	<pre> TL1=0XF3; ES=1; //打开接收中断 EA=1; //打开总中断 TR1=1; //打开计数器 } void Receive_sbuf() interrupt 4 //串口接收函数 { int receive; receive = SBUF; sbuf = receive; buff[0] = sbuf; RI = 0; } int main() //主函数 { Time0_initial(); SBUF_initial(); while (1) { BreathLed(sbuf); } } </pre>
--	--

5.10 心德

串口里面有许多新的东西，比如波特率，这的确难懂，但是假以时日，没有不能攻克的！学习工科，要舍得花时间来夯实这些理论。它们是你做实验的基础。

6.1 错误是最好的老师

中断中的常见错误：

1) 忘记开启中断寄存器

```

void init_UART(void)
{
    SCON = 0x50;
    TMOD = 0x20;
    TH1 = 0xfd;
    TL1 = 0xfd;

```

```

    TR1 = 1;
}

```

Keil编译错误提示：
无

错误分析：

这样的初始化是肯定会引起程序错误的，在运用串口通信或者中断(只要包含中断)时，在初始化定义时必须记得打开相应中断寄存器和总中断寄存器，避免有些寄存器在复位时并没有打开而引起系统出错。

错误更改：

```

void init_UART(void)
{
    SCON = 0x50;
    TMOD = 0x20;
    TH1 = 0xfd;
    TL1 = 0xfd;
    TR1 = 1;
    ES  = 1;
    EA  = 1;
}

```

2) 接收发送过程的超时或延迟问题

第一种情况：

```

void main (void)
{
    init();
    while (1)
    {
        send1("Hello world!\n");
        delay(50000);
    }
}

void send2(unsigned char dat)
{
    SBUF = dat;
    while(TI == 0);
    TI = 0;
    ES=1;
}

void send1(unsigned char *s)
{
    ES=0;
    while(*s!='\0')

```

```

    {
        send2(*s);
        s++;
    }
}

```

错误分析：

这样的在发送数据之前，先关闭了串口中断！等待发送完毕后，再打开串口中断。这样，在发送数据的等待期间内，如果收到了数据，将不能进入中断函数，也就不会保存的这个新收到的数据。

这种处理方法，就会遗漏收到的数据。

错误更改：

```

void main (void)
{
    init();
    while (TI==0)
    {
        .....
    }
}
void send2(unsigned char dat)
{
    SBUF = dat;
    while(TI == 0);
    TI = 0;
    //————
}
void send1(unsigned char *s)
{
    //————
    while(*s!='\0')
    {
        send2(*s);
        s++;
    }
}

```

第二种情况：

```

void main (void)
{
    init();
    while (TI==1)
    {

```

```

        ..... (语句一)
    }
}
void ser_int (void) interrupt 4
{
    TI=0;
    ..... (语句二)
}

```

错误分析:

虽然在发送数据之前，并没有关闭串口中断，当 $TI = 1$ 时，是可以进入中断程序的。但是，却在中断函数中，将 TI 清零！

这样，在主函数中的 `while(TI == 0);`，将永远等不到发送结束的标志。

错误更改:

```

void main (void)
{
    init();
    while (1)
    {
        .....
    }
}
void ser_int (void) interrupt 4
{
    if (TI==1)
    {
        ..... (语句一)
        TI=0;
    }
    ..... (语句二)
}

```

第三种情况:

```

void main (void)
{
    init();
    while (1)
    {
        ..... (语句一)
    }
}
void ser_int (void) interrupt 4
{
    if (TI==1)

```

```

{
    ..... (语句二)
    TI=0;
}
if(RI==1)
{
    RI=0;
    ..... (语句三)
}
..... (语句四)
}

```

错误分析：

在中断程序中，并没有区分中断的来源，反而让发送引起的中断，执行了接收中断的程序。

错误更改：

接收数据时，使用“中断方式”，清除 RI 后，用一个变量通知主函数，收到新数据。

发送数据时，也用“中断方式”，清除 TI 后，用另一个变量通知主函数，数据发送完毕。

反思与改错

附录

KEIL 的安装与调试

KEIL 软件是 KEIL 公司为 8051 单片机研制的软件编译器，下面就让我们进行 KEIL 软件的安装与调试吧。

1.KEIL 软件的安装

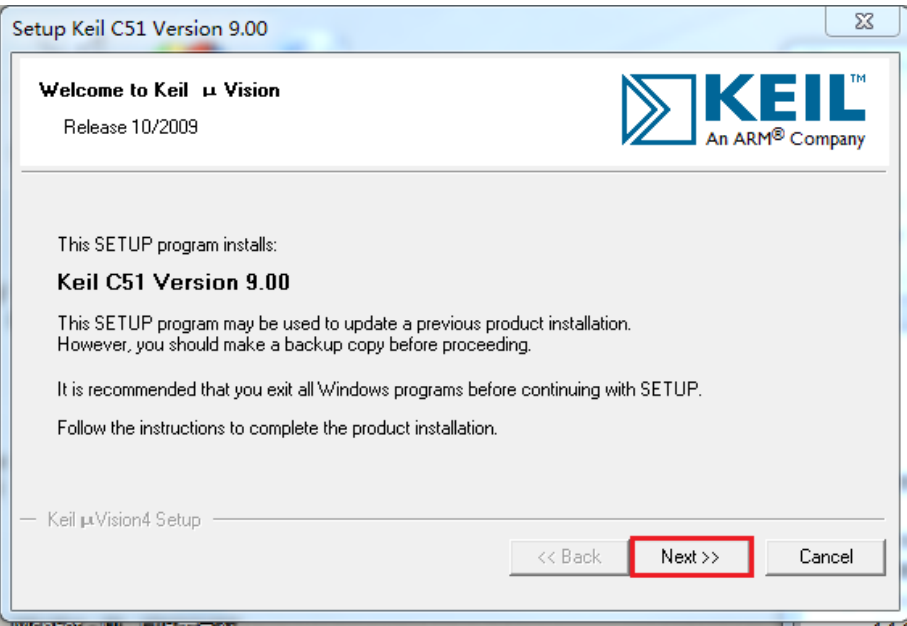
1.1 KEIL 软件的安装：

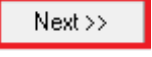
在互联网上下载 KEIL 软件安装包（含破解软件），这里笔者就不叙述了，然后解压，解压结果如图 1 所示：

 注册机	2010/8/9 16:24	文件夹	
 173绿色软件	2010/3/19 8:52	Internet 快捷方式	1 KB
 173软件下载.txt	2010/5/5 9:22	文本文档	2 KB
 C51V900.exe	2009/10/26 9:41	应用程序	31,082 KB

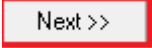
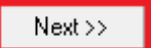
注意：可能您的解压结果和图 1 有差异，但都应该有如图 1 所示的注册机和安装程序。

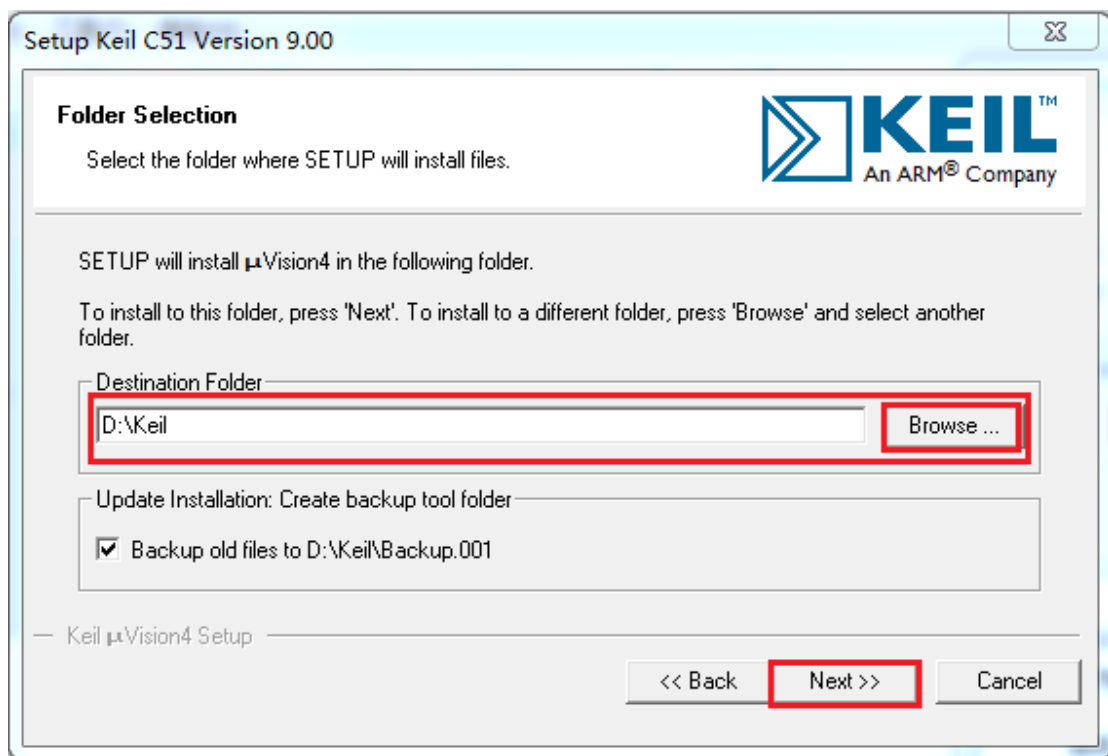
鼠标左键双击 C51V900.exe，出现如图 2 所示的画面：



鼠标左键单击，出现如图三所示的画面：

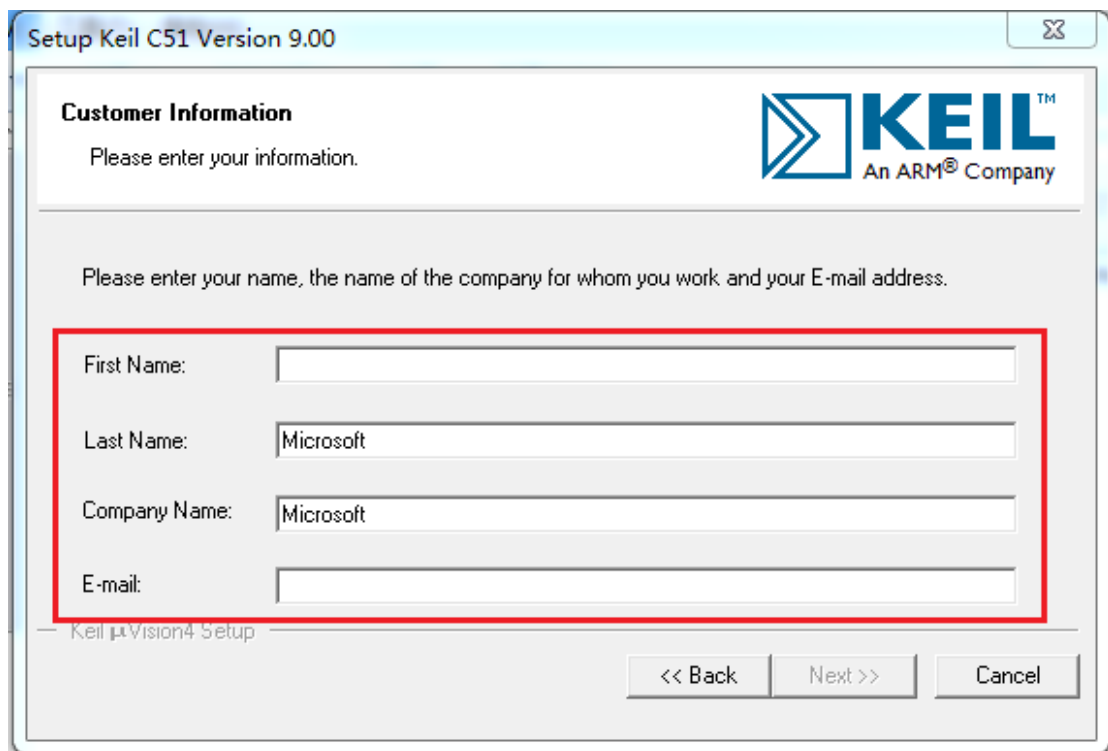


细心地读者会发现，按钮单击没有反应了，不用担心，注意到图 3 红色边框内的英文句子 “I agree to all the terms of the preceding License Agreement”，我们只需用鼠标左键单击红色方形框即可，此时变成了，接下来鼠标左键单击，得到如图 4 所示的画面：

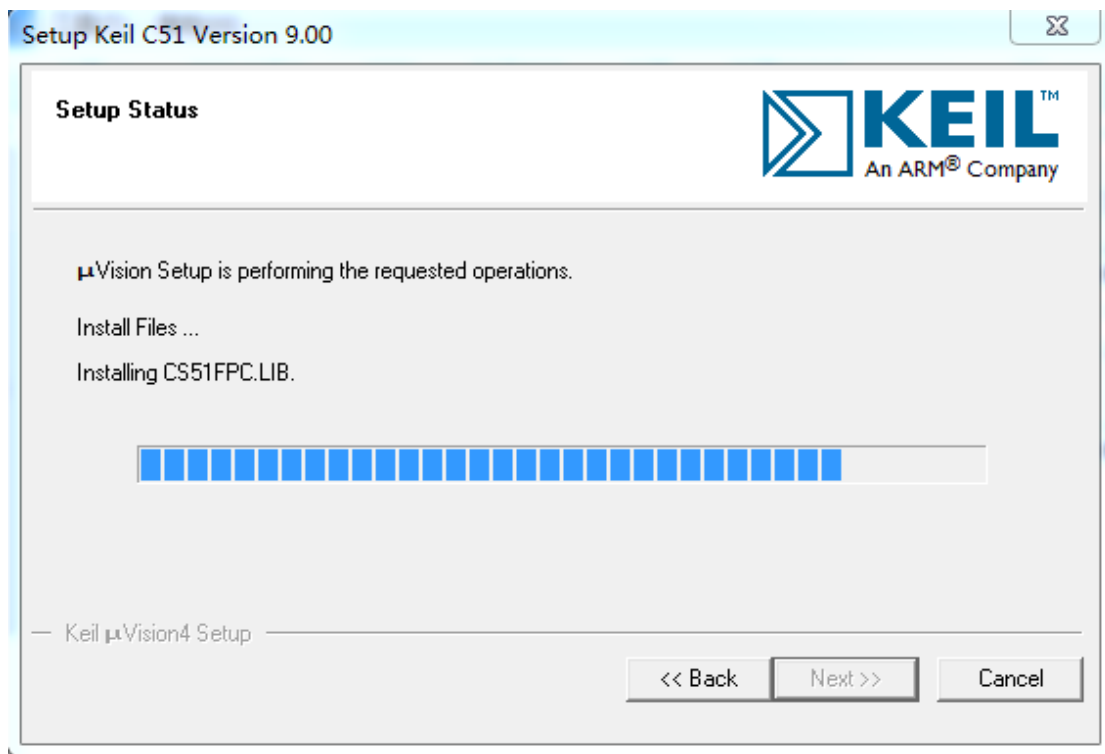


其中，矩形框内 D:\Keil 表示 KEIL 软件的安装地址，如需改变，鼠标左键单击 **Browse ...** 按钮即可，这里不再叙述。

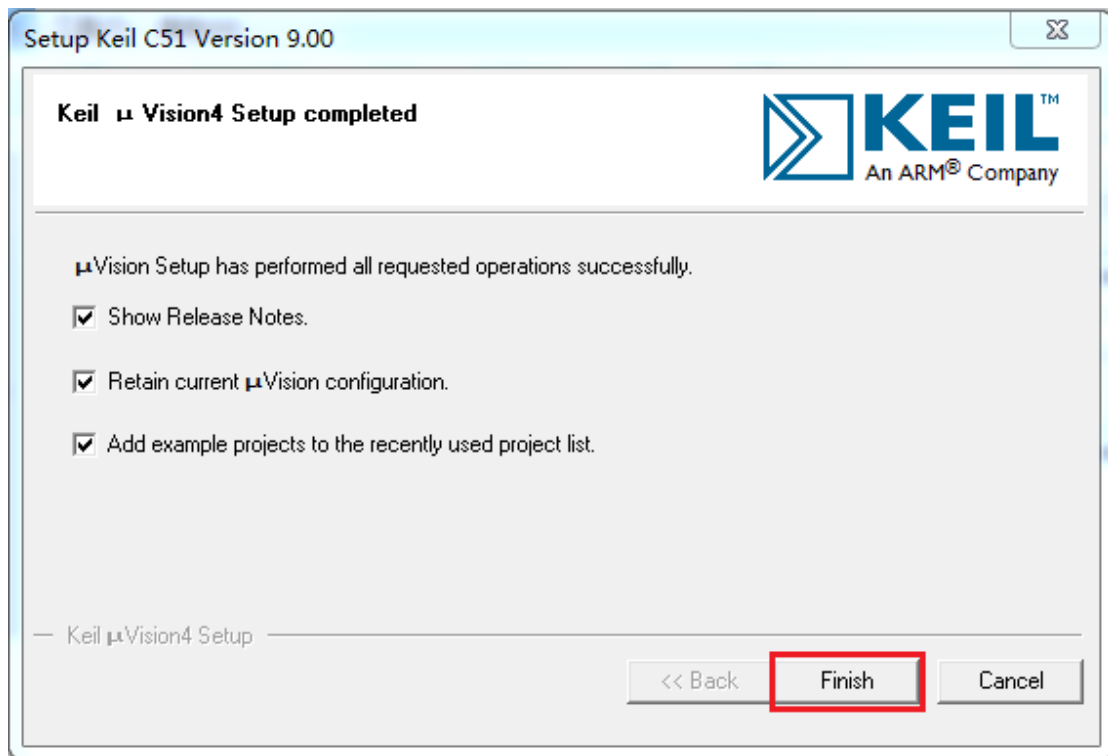
鼠标左键单击 **Next >>** 按钮，进入如图 5 所示的画面：



我们注意到 **Next >>** 按钮再一次无法继续点击了，我们只需填好矩形框内的信息即可，鼠标左键单击 **Next >>** 按钮，进入安装进程，如图 6 所示：



安装进程结束后，出现如图 7 所示的画面：



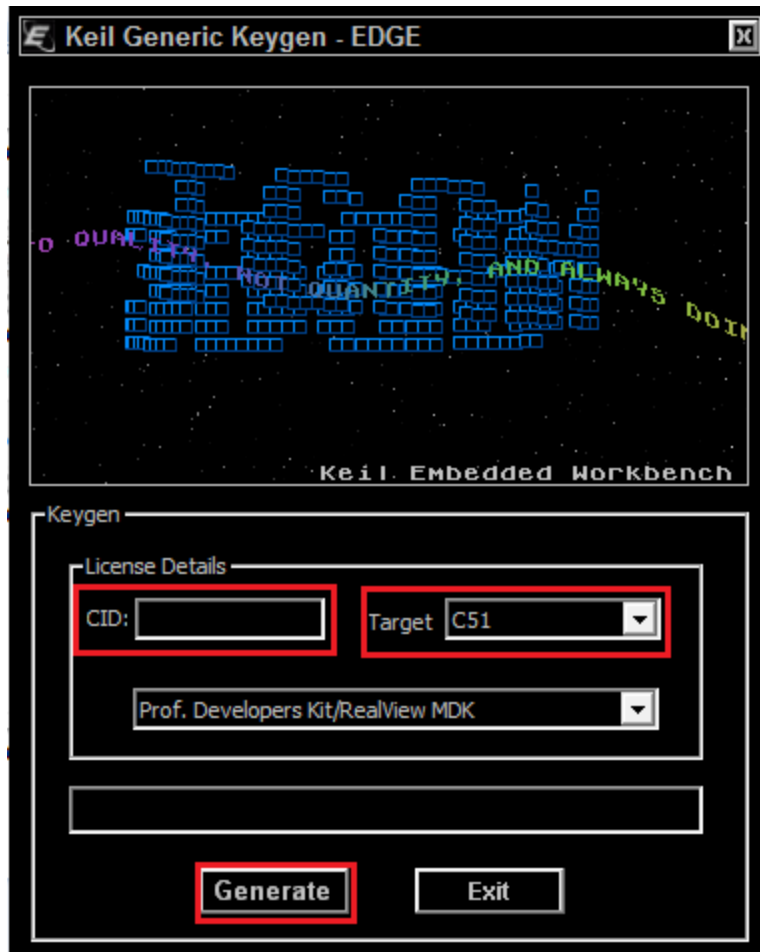
鼠标左键单击 **Finish** 按钮，结束 KEIL 软件的安装。

1.2 KEIL 软件的破解

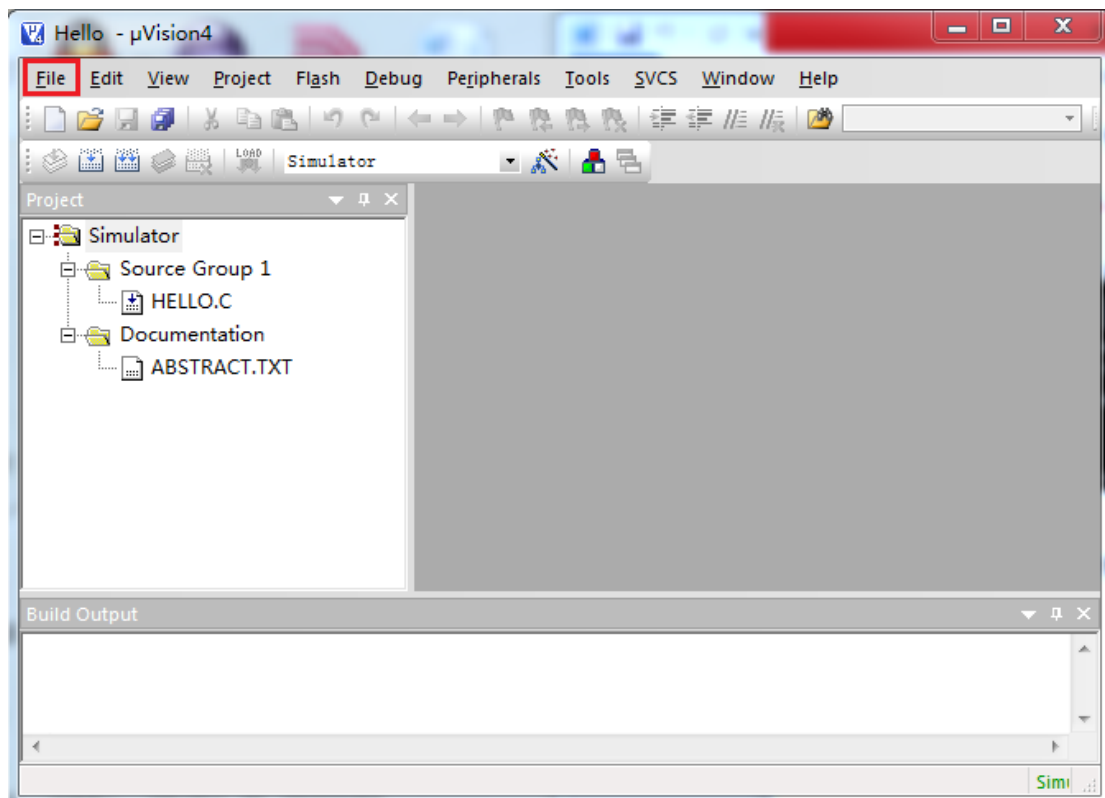
打开 **注册机** 文件夹，结果如图 8 所示：

 KEIL_Lic.exe	2008/2/27 17:42	应用程序	18 KB
 注册机使用方法.txt	2009/12/13 1:19	文本文档	1 KB

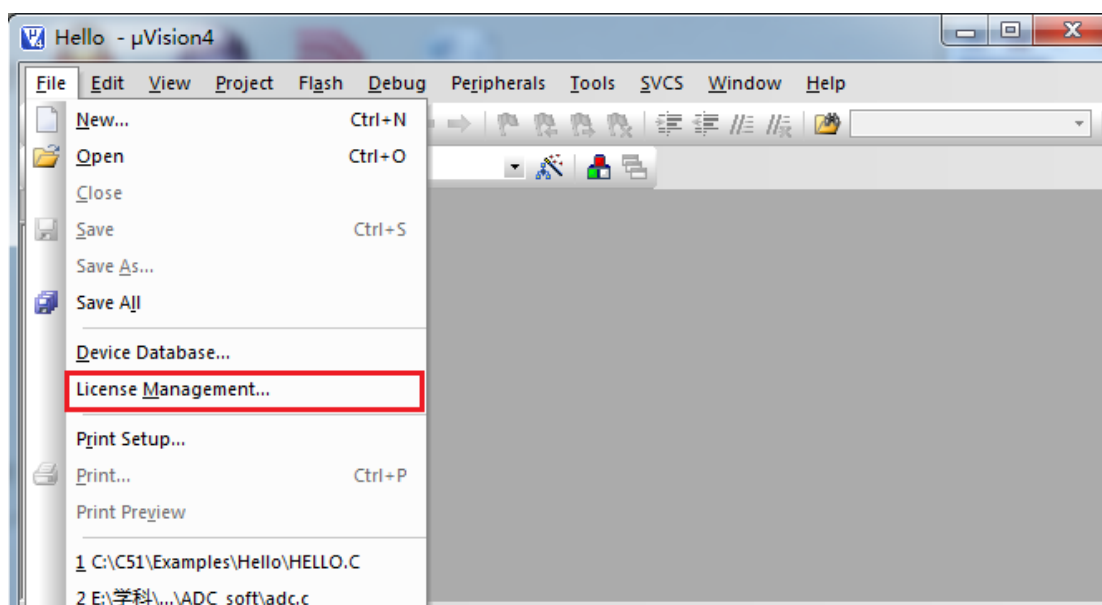
鼠标左键双击 **KEIL_Lic.exe**，进入如图 9 所示的画面：



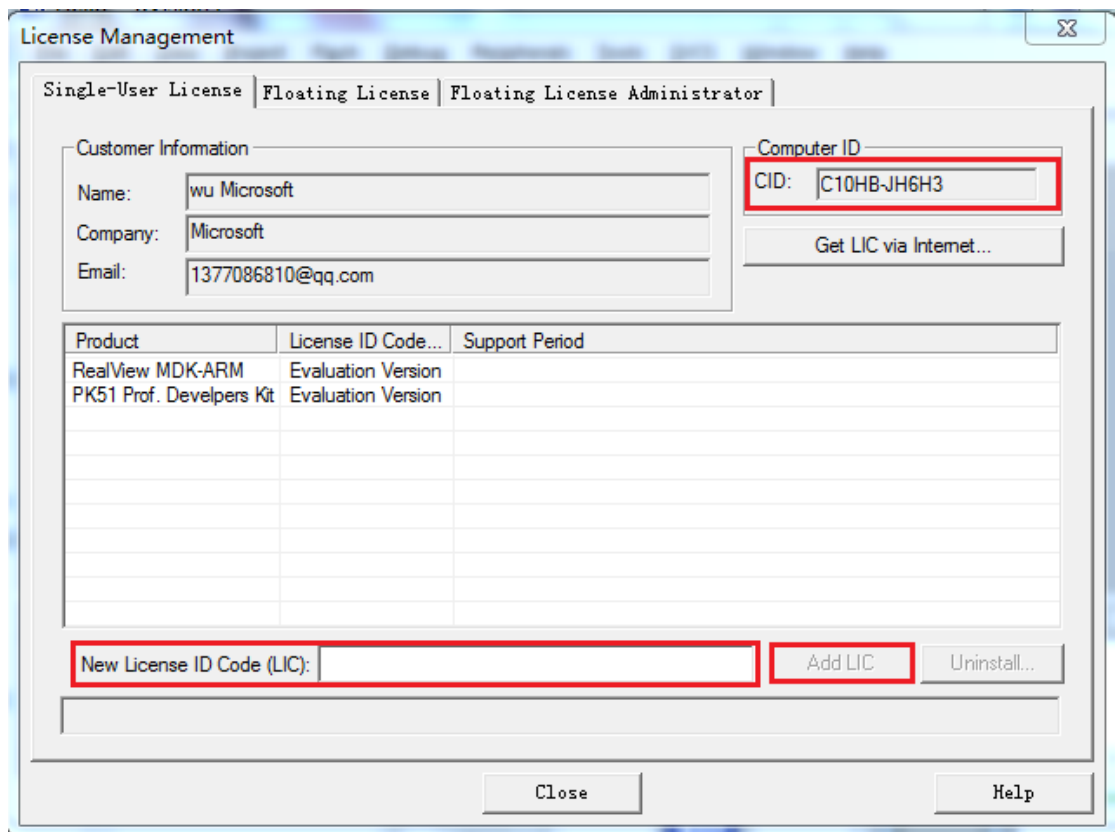
鼠标左键双击桌面的图标，打开 KEIL 软件，打开结果如图 10 所示：



鼠标左键单击 **File**，出现如下图所示的下拉列表：



鼠标左键单击 **License Management...**，出现如图 11 的画面：



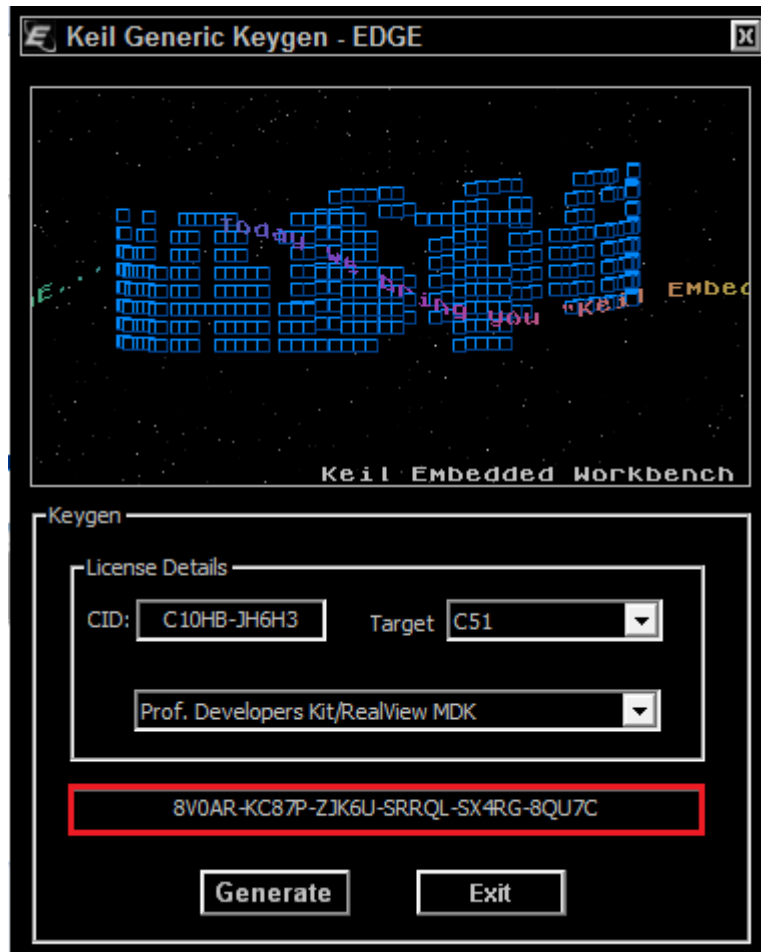
The image shows a 'License Management' dialog box with three tabs: 'Single-User License', 'Floating License', and 'Floating License Administrator'. The 'Single-User License' tab is active. It contains a 'Customer Information' section with fields for Name (wu Microsoft), Company (Microsoft), and Email (1377086810@qq.com). To the right is a 'Computer ID' section with a 'CID' field containing 'C10HB-JH6H3', which is highlighted with a red box. Below this is a 'Get LIC via Internet...' button. In the center is a table with three columns: 'Product', 'License ID Code...', and 'Support Period'. The table contains two rows: 'RealView MDK-ARM' and 'PK51 Prof. Developers Kit', both with 'Evaluation Version' in the 'License ID Code...' column. At the bottom, there is a 'New License ID Code (LIC):' field, an 'Add LIC' button (highlighted with a red box), and an 'Uninstall...' button. The dialog has 'Close' and 'Help' buttons at the bottom right.

Product	License ID Code...	Support Period
RealView MDK-ARM	Evaluation Version	
PK51 Prof. Developers Kit	Evaluation Version	

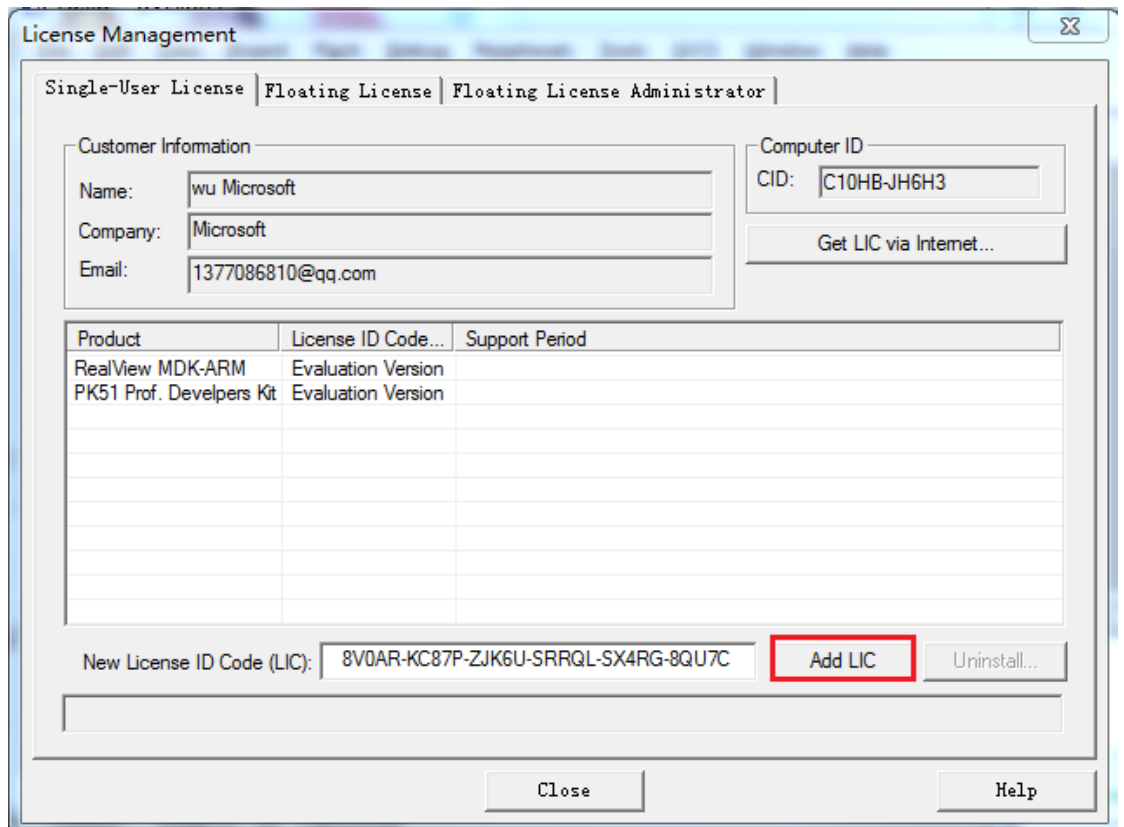
将 CID 框内的 CID 号复制到注册机的 CID 框中，点击

Generate

按钮，出现如下图所示的画面：



将红色框中的号码复制到 中，如下图所示：

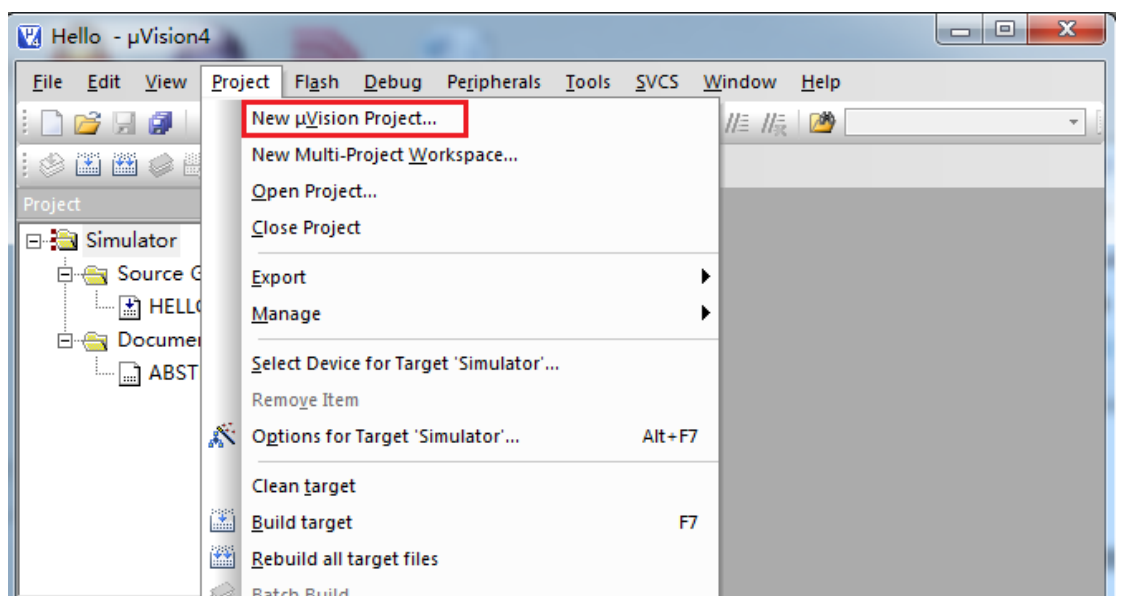


鼠标左键单击 **Add LIC** 按钮，并关闭窗口，完成破解。

2 KEIL 软件的调试

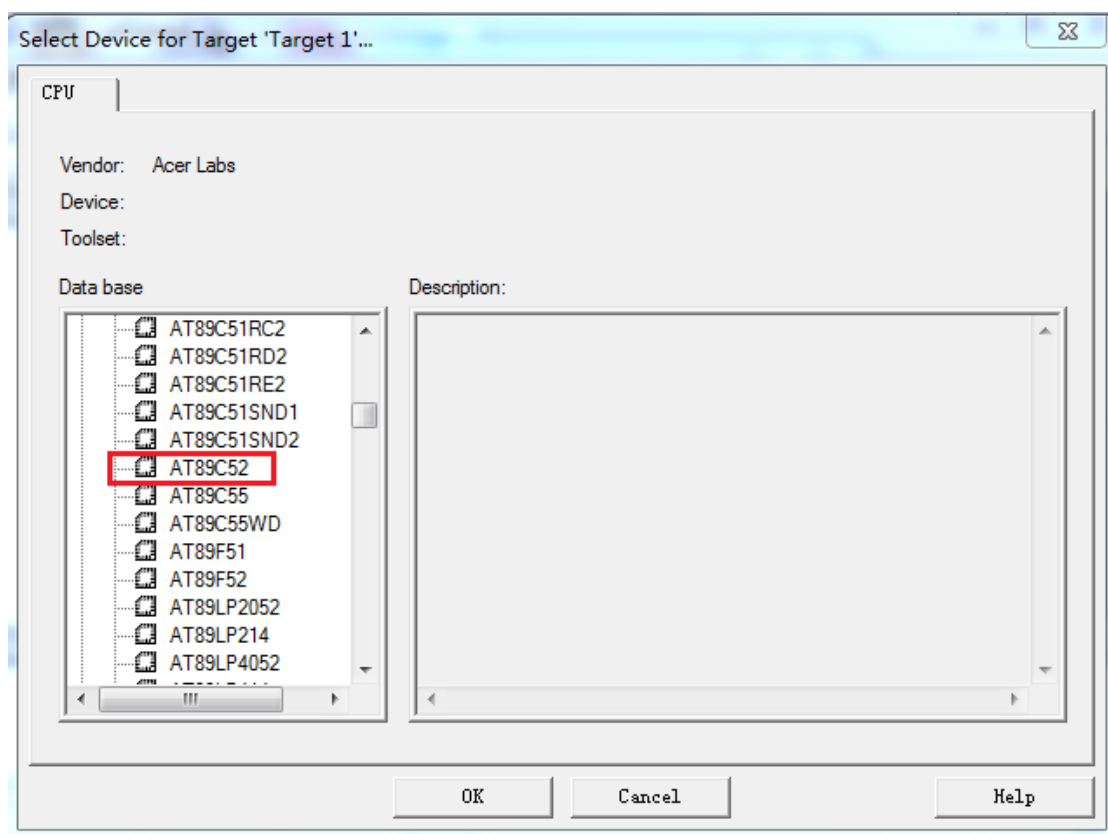
2.1 建立自己的第一个工程

鼠标左键单击 **Project** 选项，出现如下图所示的画面：

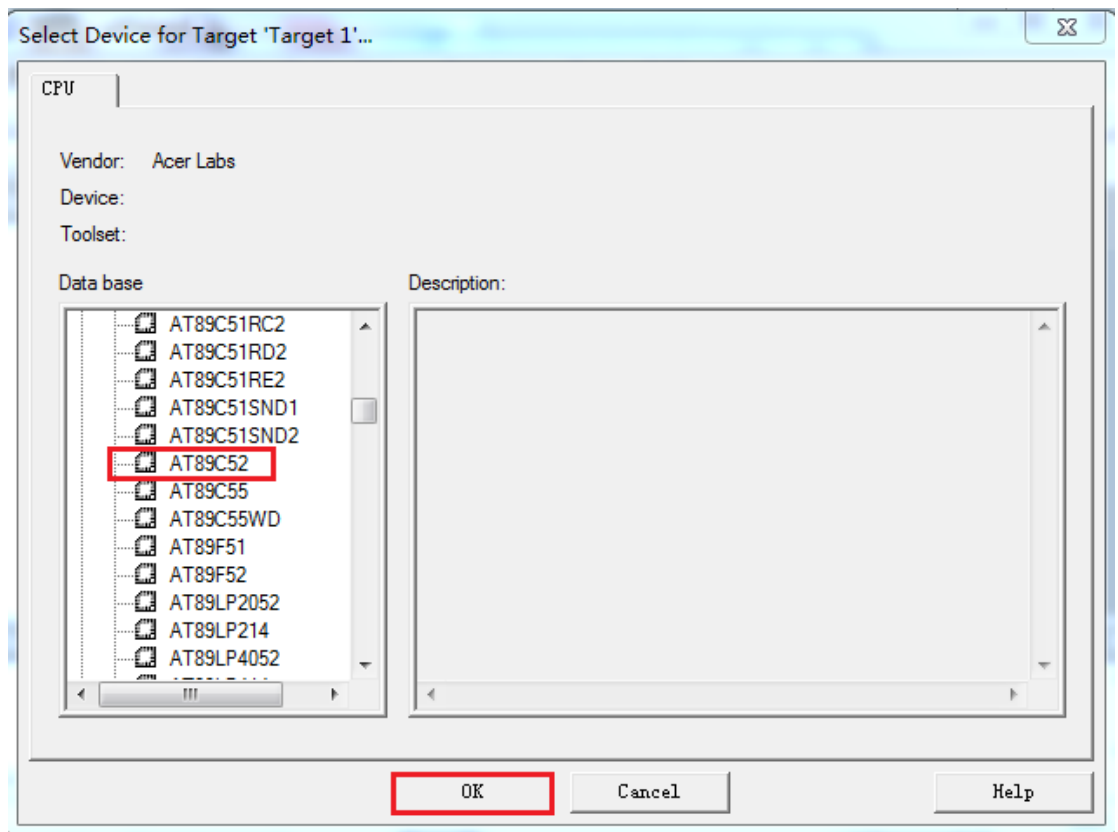


鼠标左键单击 **New uVision Project...**，输入工程名并选好所建

工程的地址（建议单独放在一个文件夹内），单击确定后出现下图所示的画面：



选择列表中  Atmel 选项下的  AT89C52 选项，并点击  ，如下图画面所示：



这样，一个新的工程就建好了。

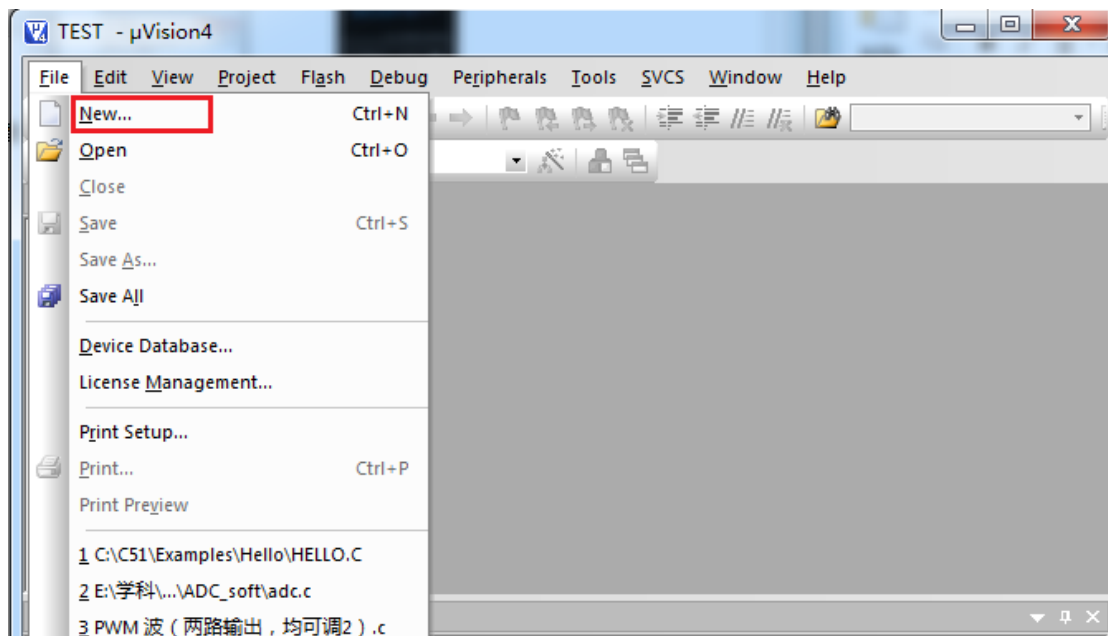
注意：有的同学在建工程的时候，发现列表选项里没有



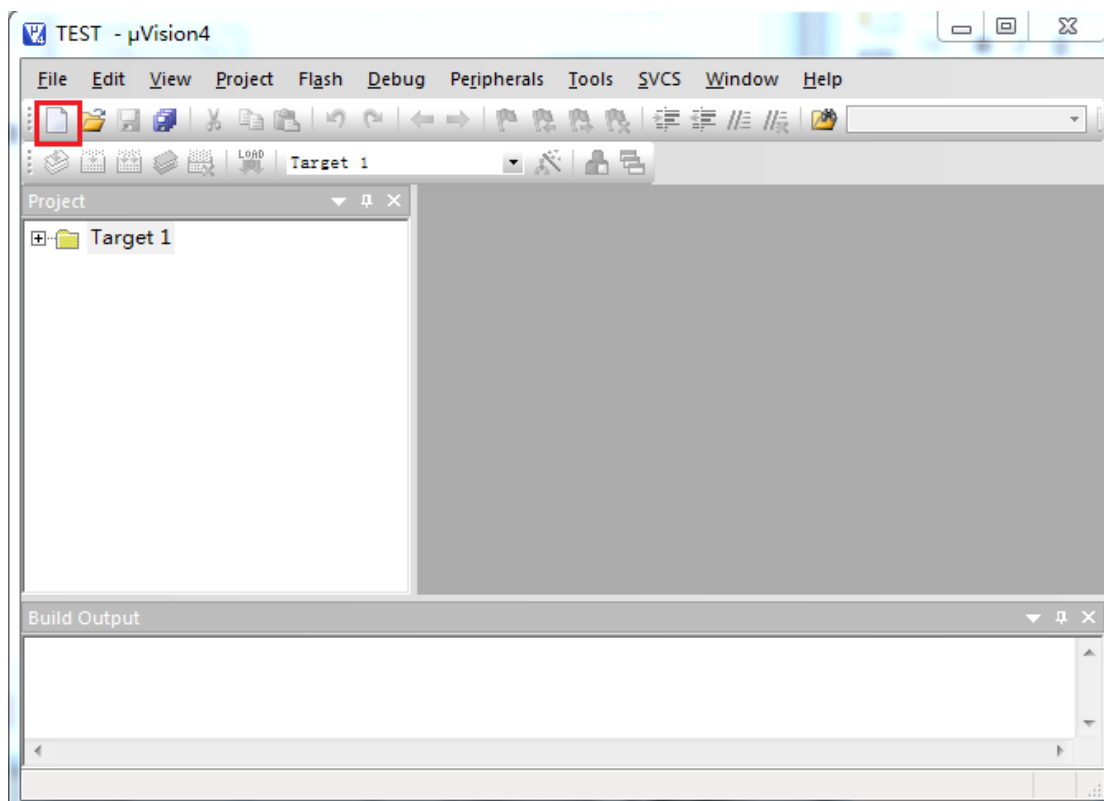
，那么有可能是 KEIL 软件下载错误了，下载的是 KEIL for ARM ,而不是 KEIL for 8051，请重新认真的下载正确的软件。

2.2 建立自己的第一个文件并添加到所建工程中

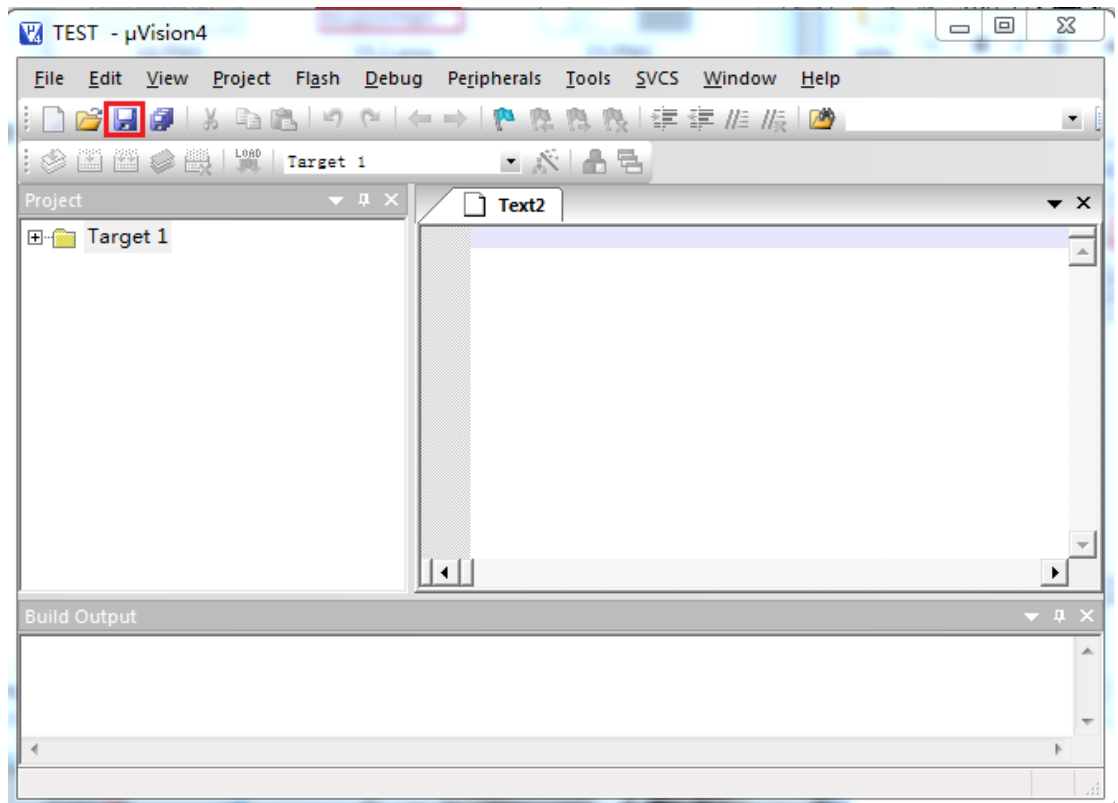
鼠标左键单击 **File**，出现下拉列表，单击 **New...**，如下图所示：



也可以直接点击 ，如下图所示：

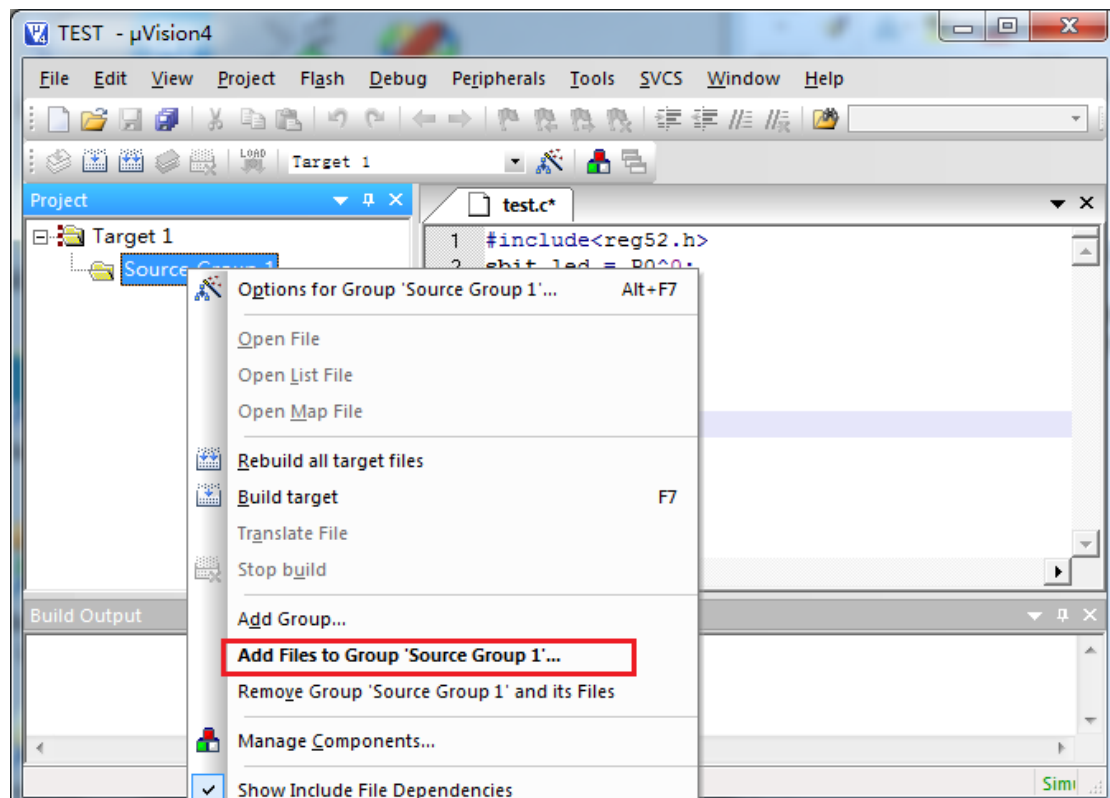


单击 ，将文件保存，如下图所示：



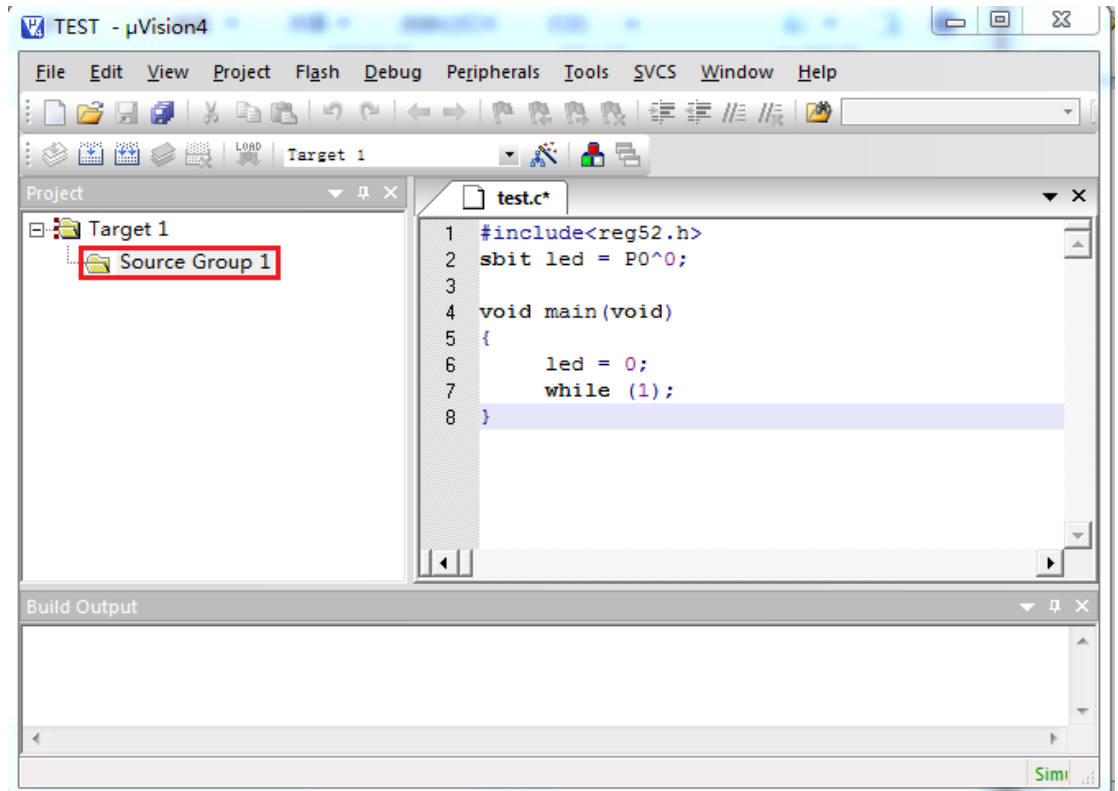
注意：在保存文件的时候保存格式为.c 文件，需在文件名后面加.c 后缀。

保存完成后，在所建文件中输入代码，并肩文件添加到所建工程中标，如下图所示：



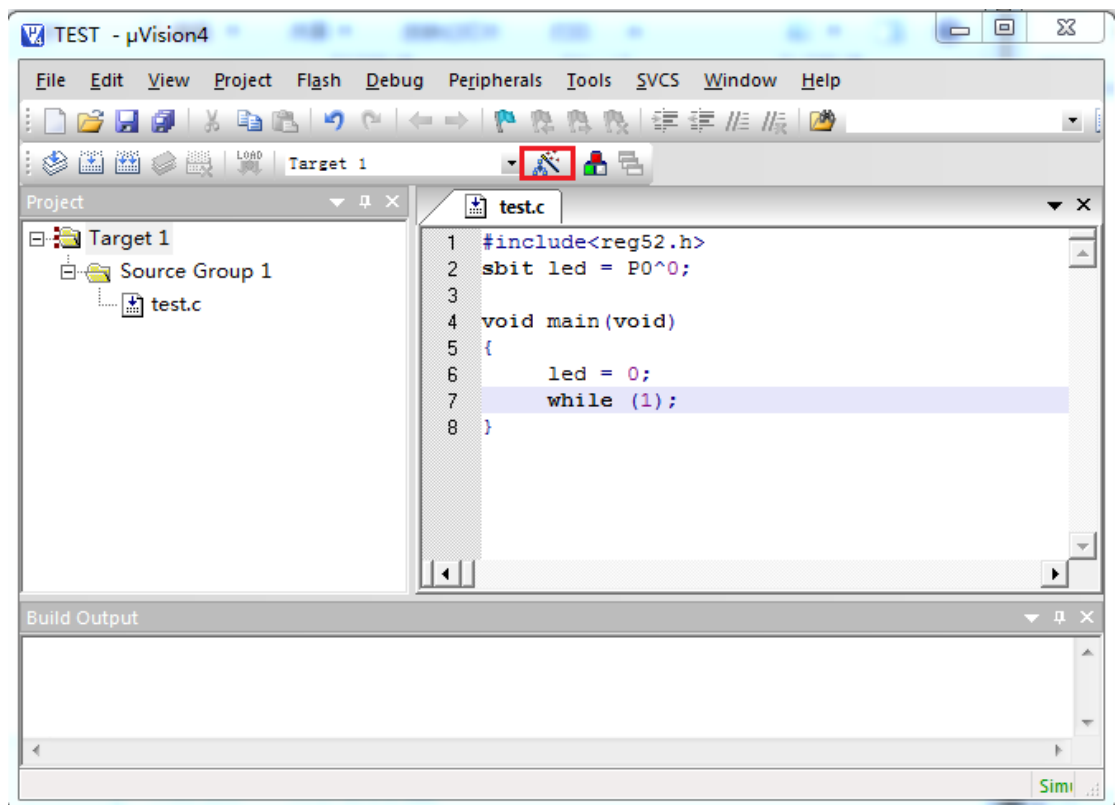
鼠标右键单击 **Add Files to Group 'Source Group 1'...**，进入添加目录，添加所编写的 C 文件。

添加文件的过程也可直接鼠标左键双击 **Source Group 1** 即可，如下图所示：

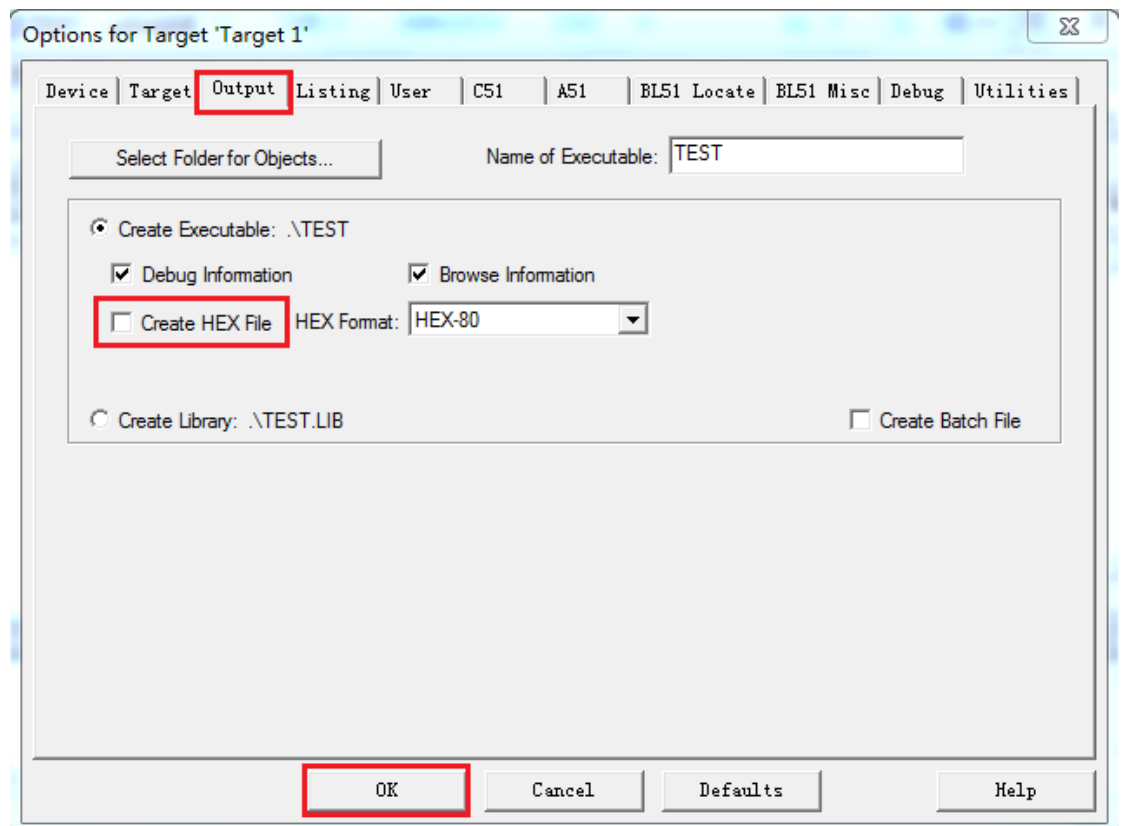


2.3 调试并下载自己的第一个程序

鼠标左键单击，如下图所示：

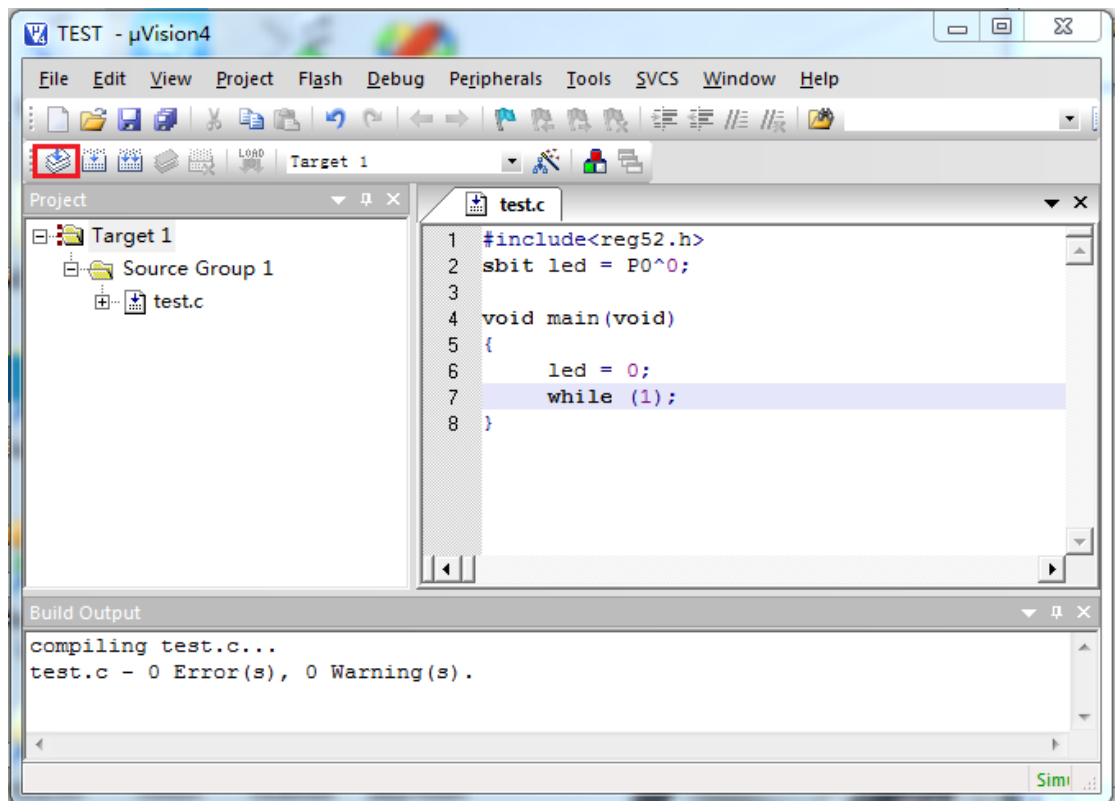


在出现的画面中鼠标左键单击 **Output**，在 ☐ **Create HEX File** 中打勾，单击 **OK** 完成，如下图所示：

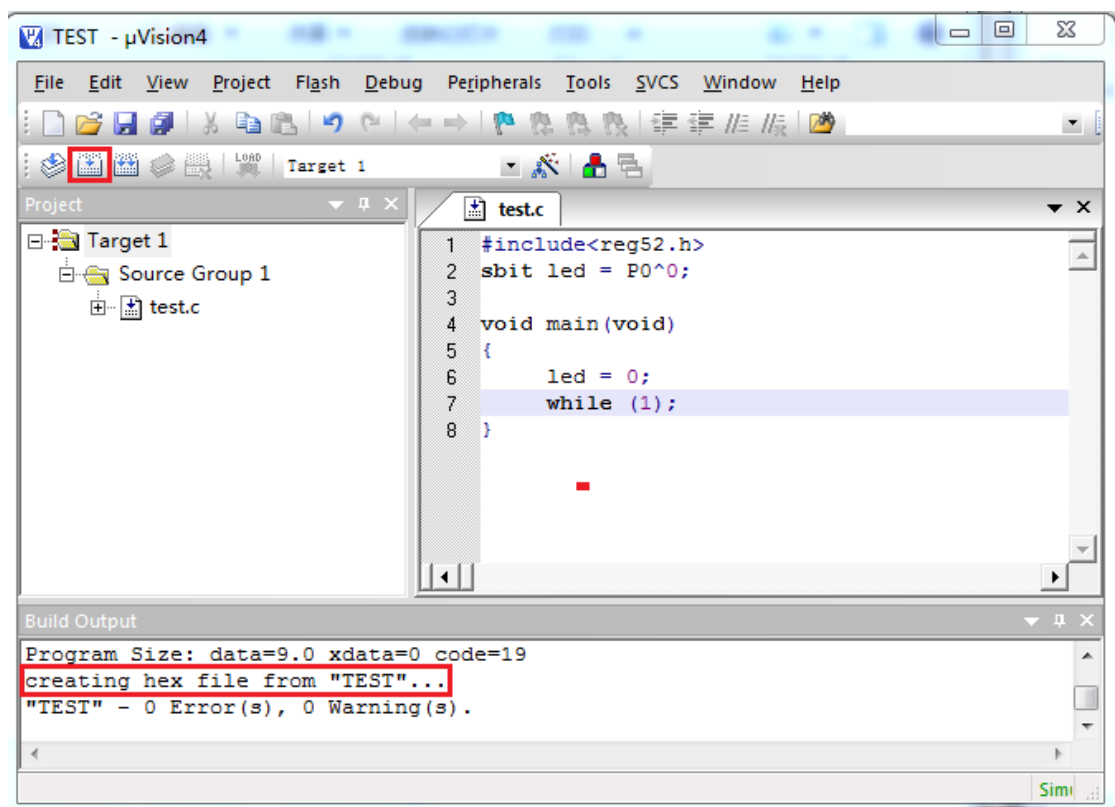


注意：只有新建工程时才需要从新配置，否则无需配置。

鼠标左键单击  进行调试，画面如下图所示：



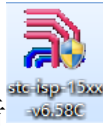
鼠标左键单击进行编译，如下图所示：



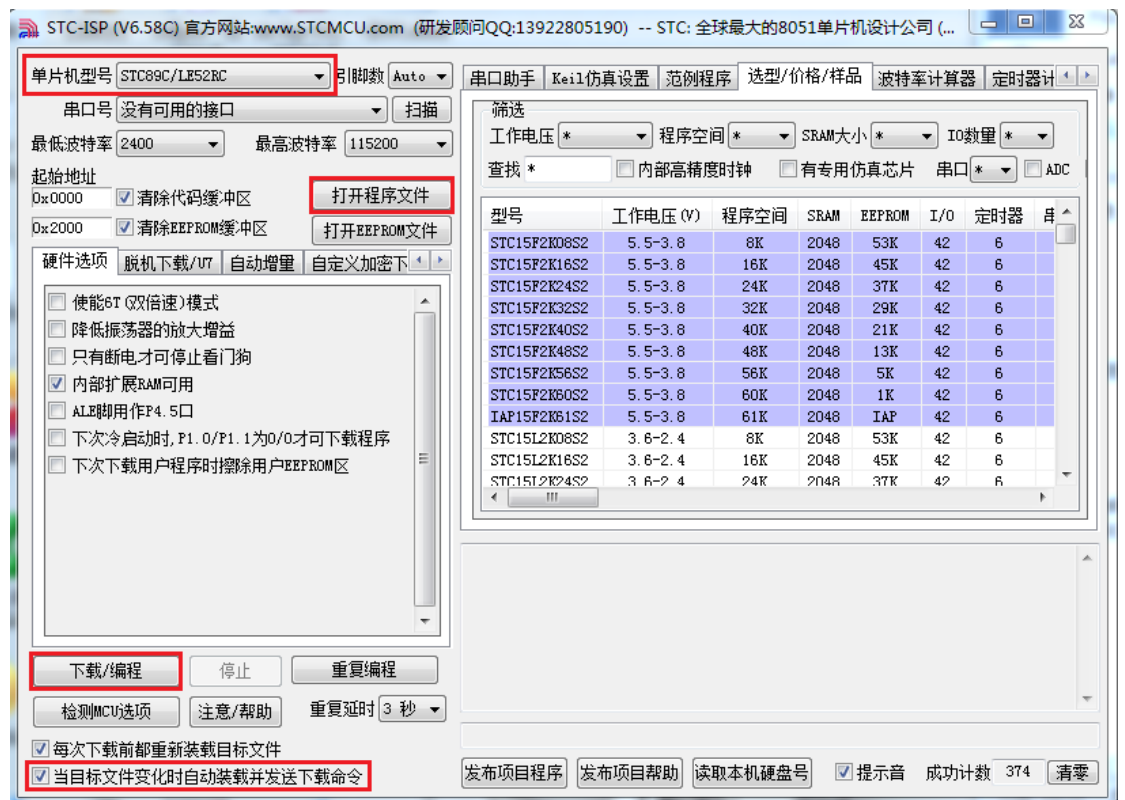
注意：在下面的信息框中，一定会出现

creating hex file from "TEST"...，这表明，已经成功创建

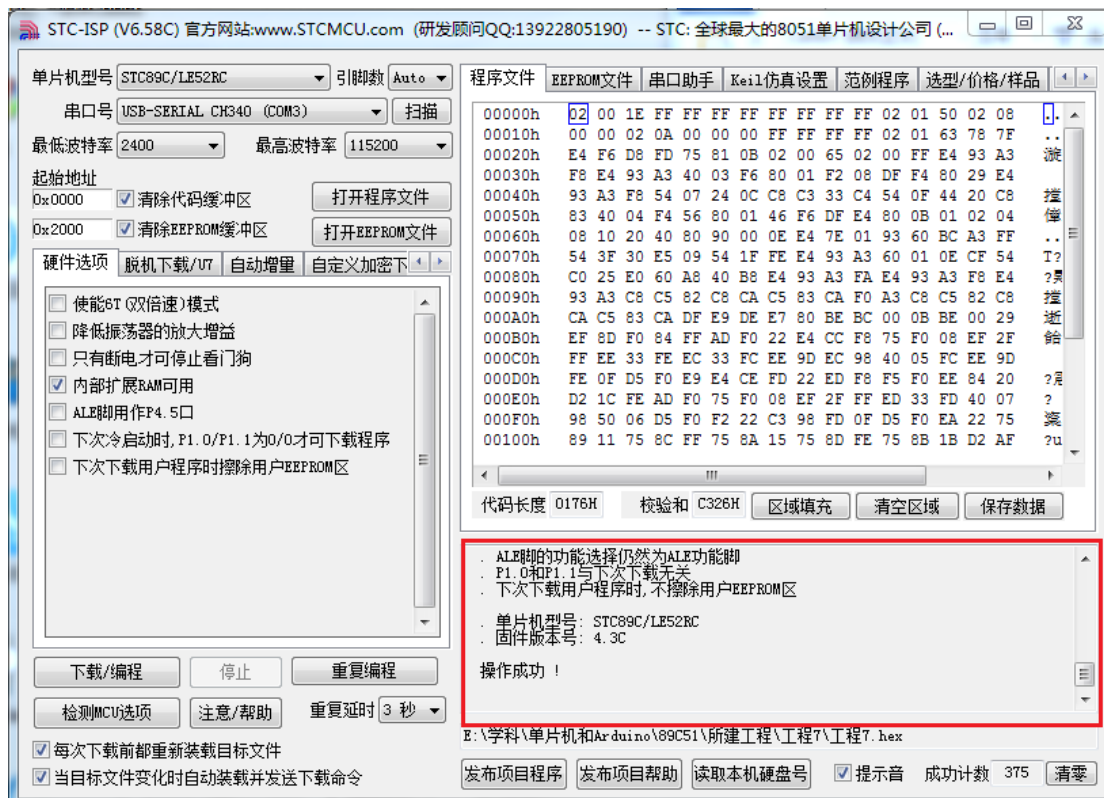
HEX 文件，如果没有，可能是前面步骤有错。



鼠标左键双击 打开 HEX 文件下载工具 STC-ISP，如图所示：



鼠标左键单击 单片机型号 STC89C/LE52RC，在下拉列表中选择单片机型号 STC89C/LE52RC，鼠标左键单击 打开程序文件，进入选择界面，选择已经生成的 HEX 文件，在单片机与电脑已连接的前提下，鼠标左键单击 下载/编程，稍微等待后，显示下载成功，如下图画面所示：



注意：当吧此处 ☒ 当目标文件变化时自动装载并发送下载命令 的勾点上时，则只需在每一次打开 STC-ISP 时点击 **打开程序文件**，并添加 HEX 文件，之后都不需要再添加了，直接单击 **下载/编程** 即可。